

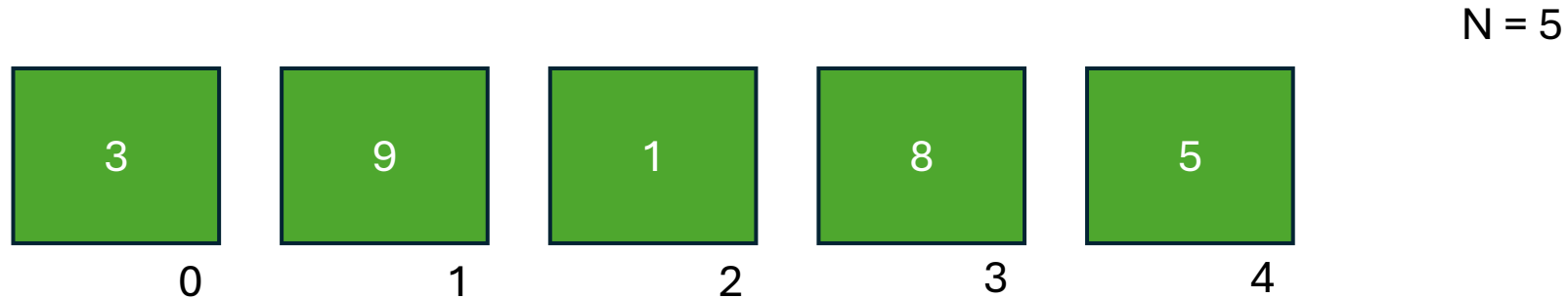
ALGORITMOS DE ORDENAMIENTO BÁSICOS

Intercambio – Selección – Inserción - Burbujeo

Profesor: Ing. Weber Federico

Ordenamiento por Intercambio [Exchange Sort]

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```

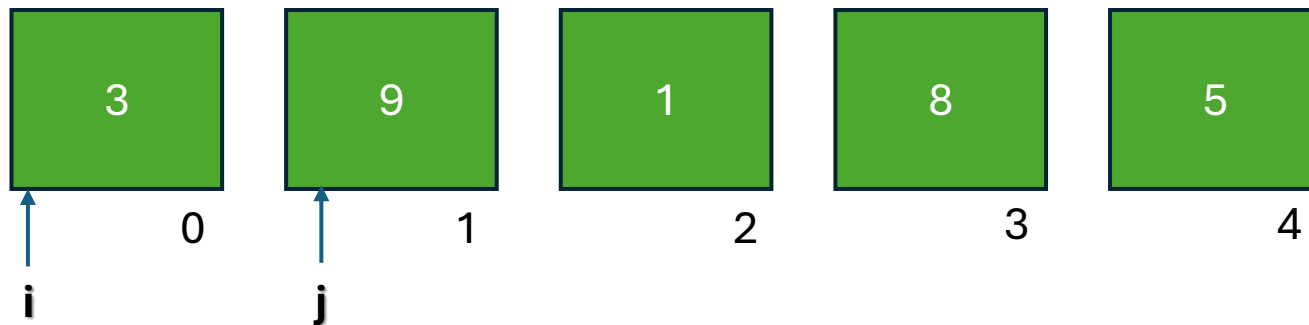


Consiste en recorrer el vector repetidamente y *comparar un elemento con todos los demás*, intercambiándolos si están fuera de orden.

En una implementación típica, el algoritmo compara el primer elemento con cada uno de los restantes y, si encuentra uno menor, los intercambia; luego procede con el segundo elemento, y así sucesivamente. De este modo, los elementos más pequeños (o mas grandes) van "pasando" hacia el inicio de la lista en cada iteración.

Ordenamiento por Intercambio [Exchange Sort]

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



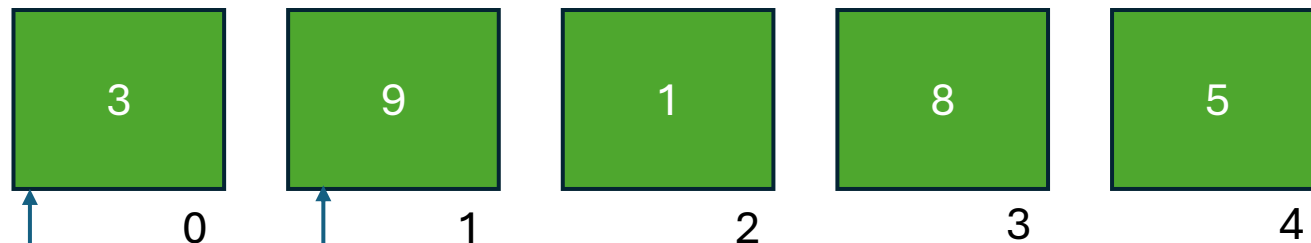
$N = 5$

i comienza en cero

j comienza en i+1

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



$N = 5$

- If (3 > 9)
 - intercambiar;

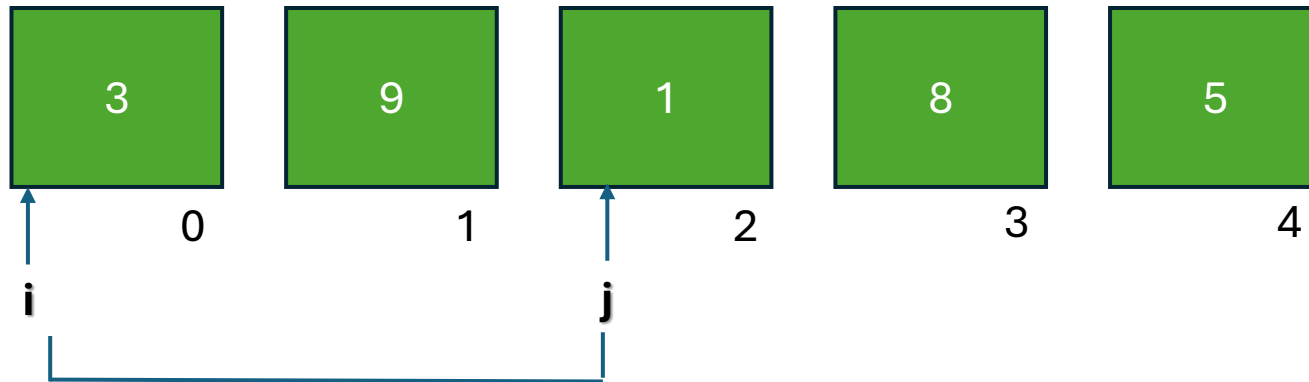
Comparación: 1

swaps: 0

Recorrido: 1

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

- If (3 > 1)
 - intercambiar;

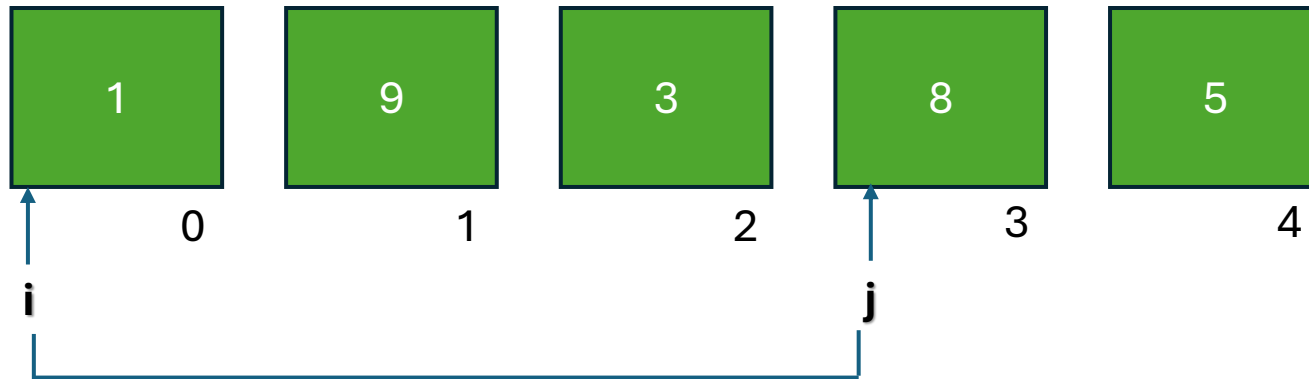
Comparación: 2

swaps: 1

Recorrido: 1

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

- If (1 > 8)
 - intercambiar;

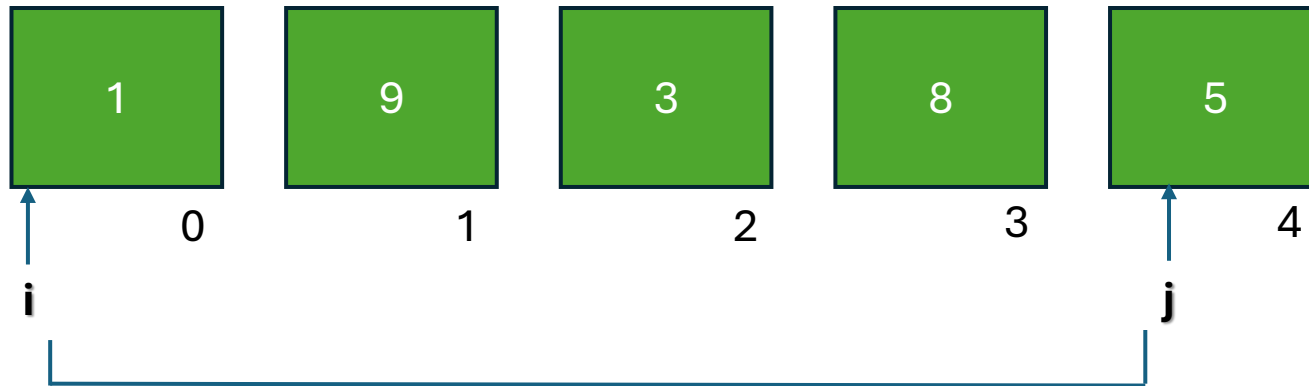
Comparación: 3

swaps: 1

Recorrido: 1

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



- If (1 > 5)
 - intercambiar;

Comparación: 4

swaps: 1

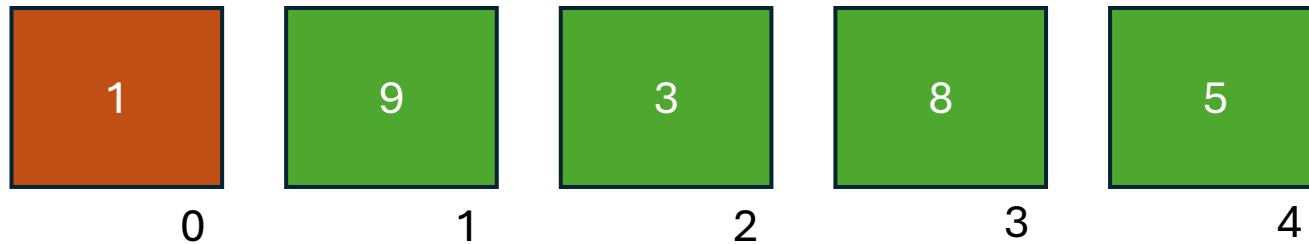
Recorrido: 1

$N = 5$

***j* recorre N-1 elementos**

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



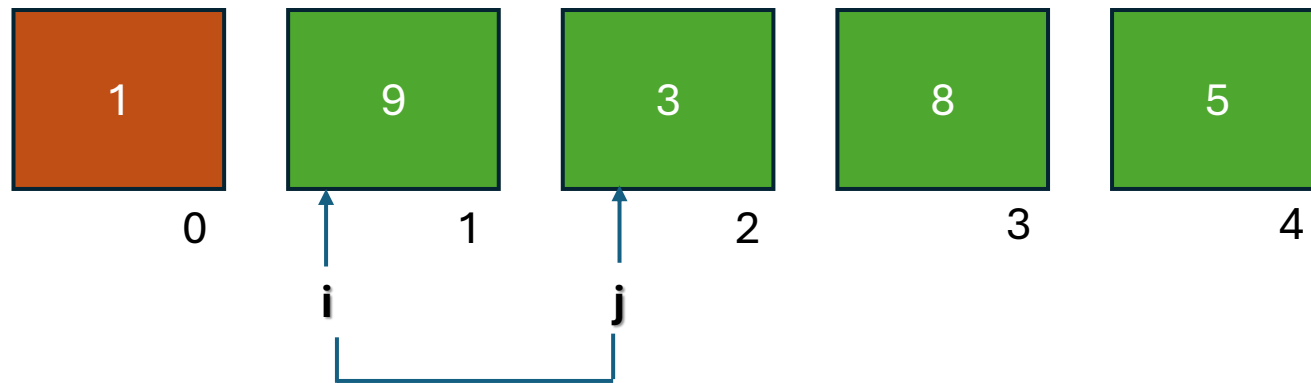
$N = 5$

Comparación: 4
swaps: 1
Recorrido: 1

En la primer vuelta se realizaron **(n-1)** comparaciones

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

- If (9 > 3)

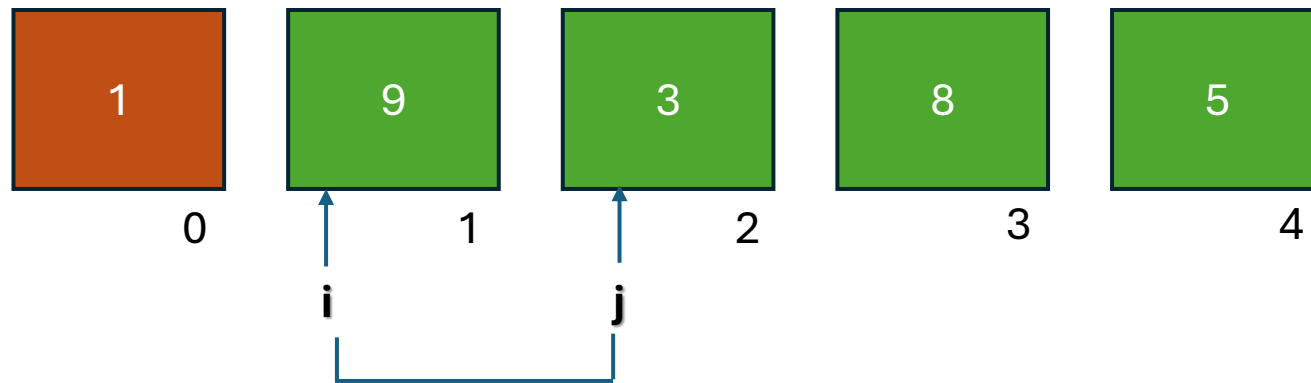
Comparación: 1

swaps: 0

Recorrido: 2

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

- If (9 > 3)
 - intercambiar;

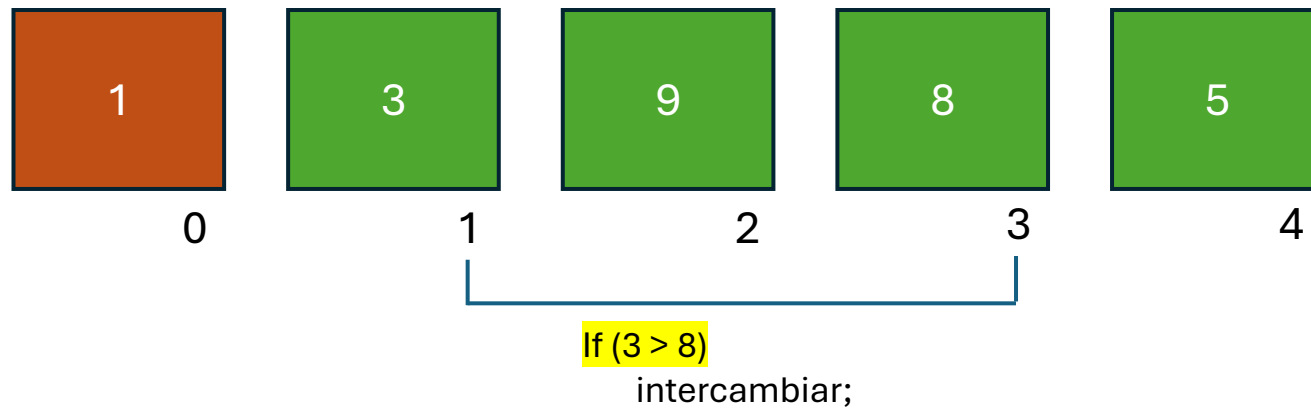
Comparación: 1

swaps: 1

Recorrido: 2

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



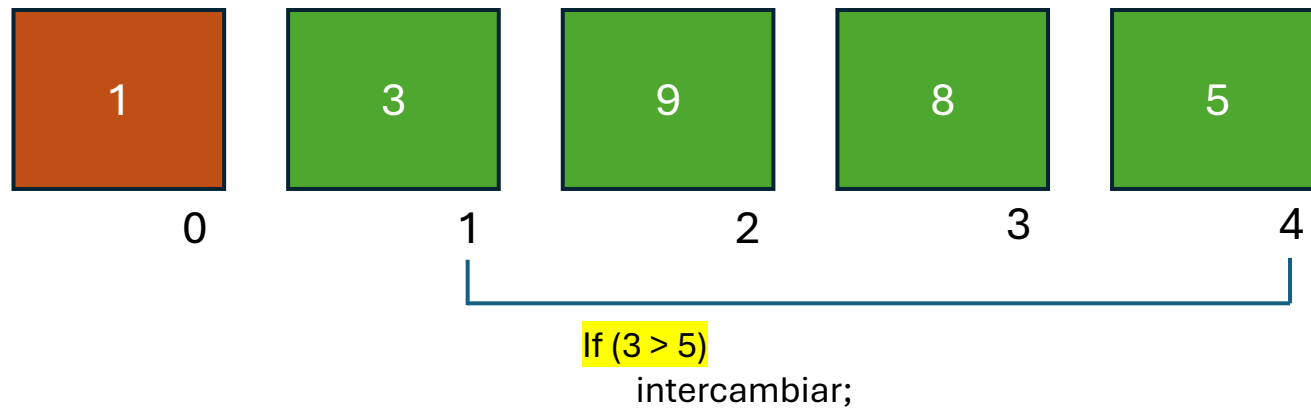
Comparación: 2

swaps: 1

Recorrido: 2

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

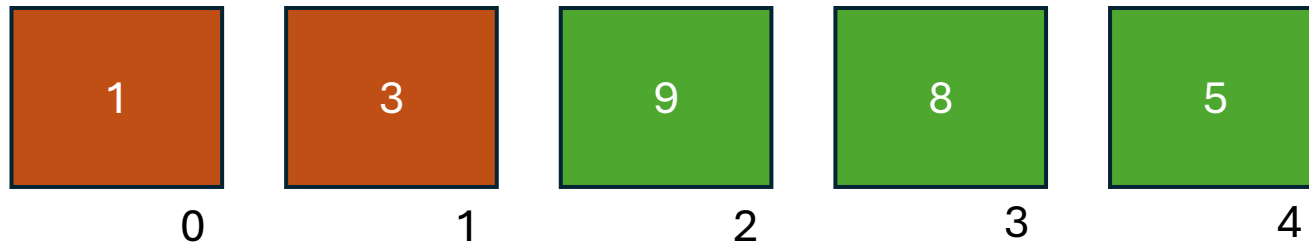
Comparación: 3

swaps: 1

Recorrido: 2

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

Comparación: 3

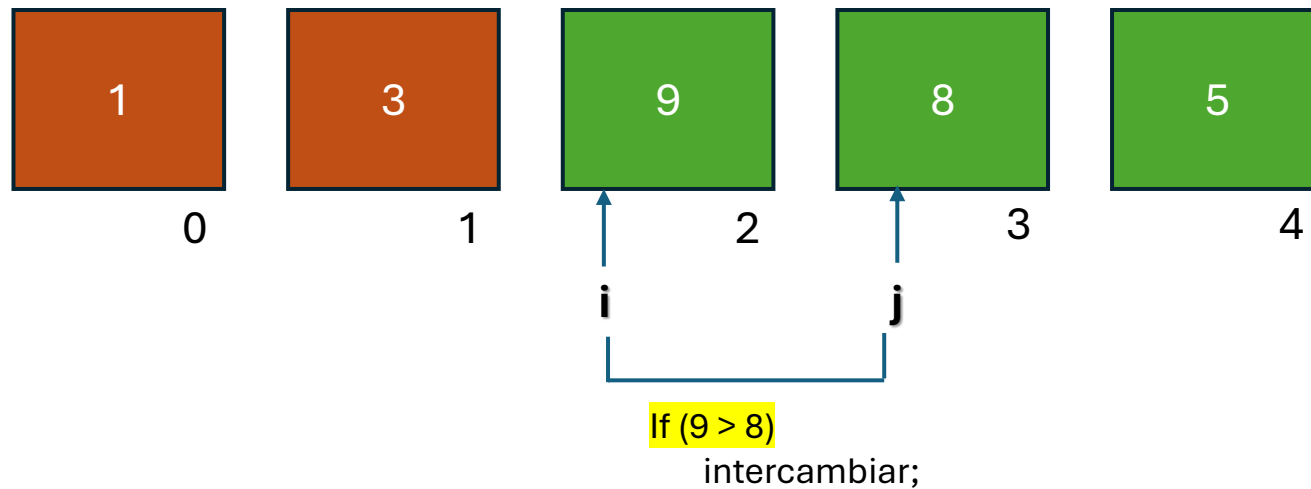
swaps: 1

Recorrido: 2

En la segunda vuelta se realizaron **(n-2)** comparaciones

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



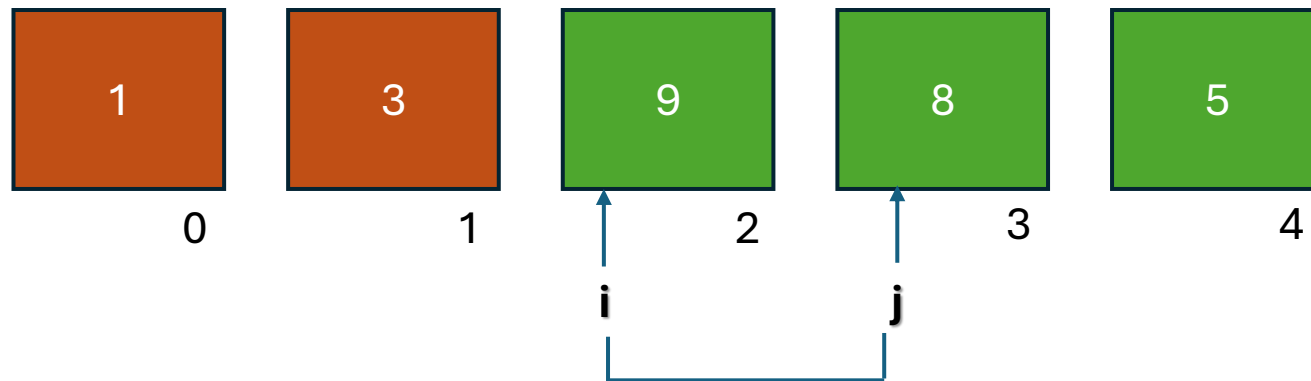
Comparación: 1

swaps: 0

Recorrido: 3

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

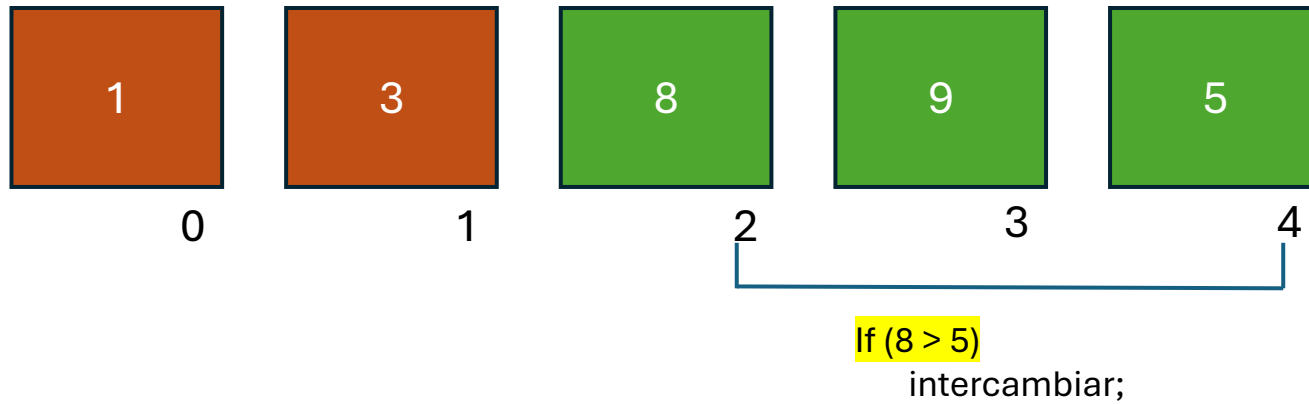
Comparación: 1

swaps: 1

Recorrido: 3

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



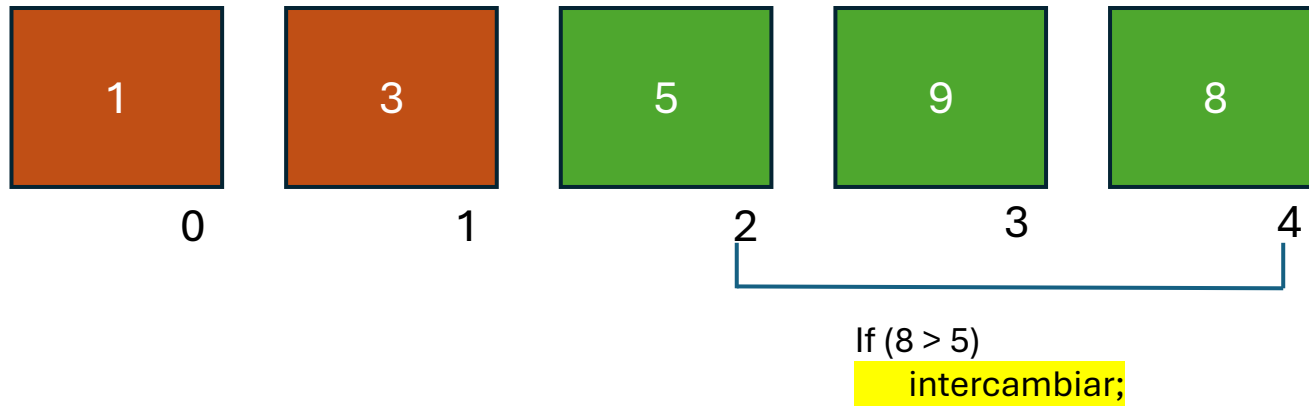
Comparación: 2

swaps: 1

Recorrido: 3

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



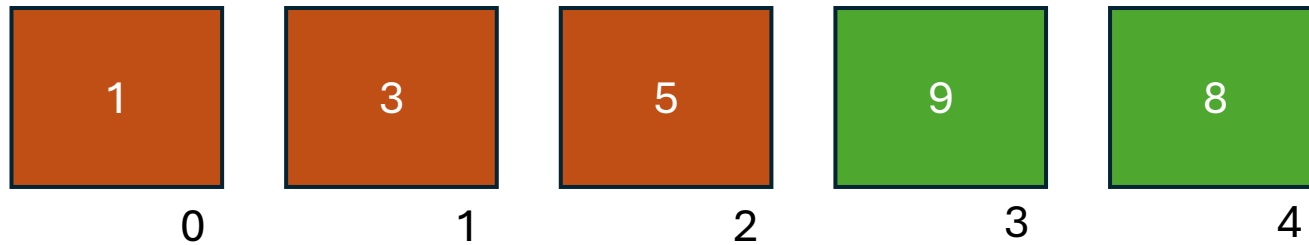
Comparación: 2

swaps: 2

Recorrido: 3

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

Comparación: 2

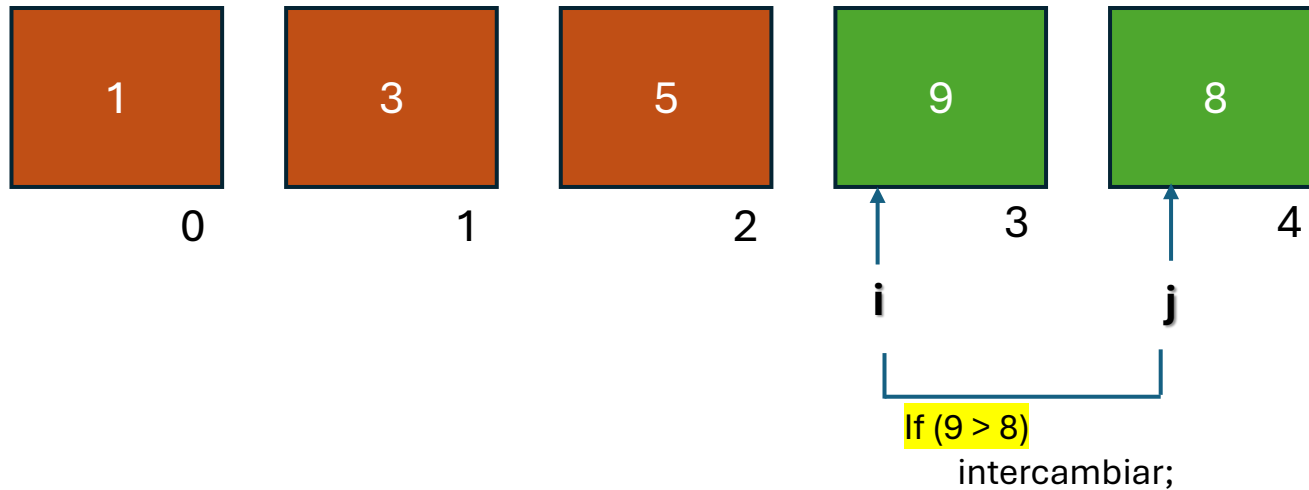
swaps: 2

Recorrido: 3

En la tercer vuelta se realizaron **(n-3)** comparaciones

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



N = 5

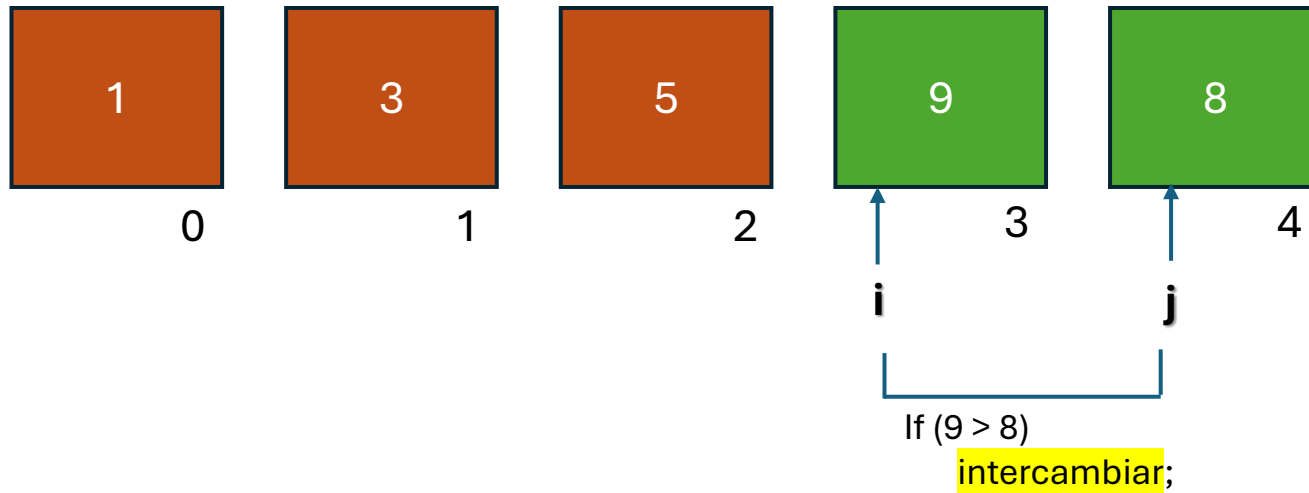
Comparación: 1

swaps: 0

Recorrido: 4

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



$N = 5$

i recorre $N-1$ elementos
j recorre $N-1-i$ elementos

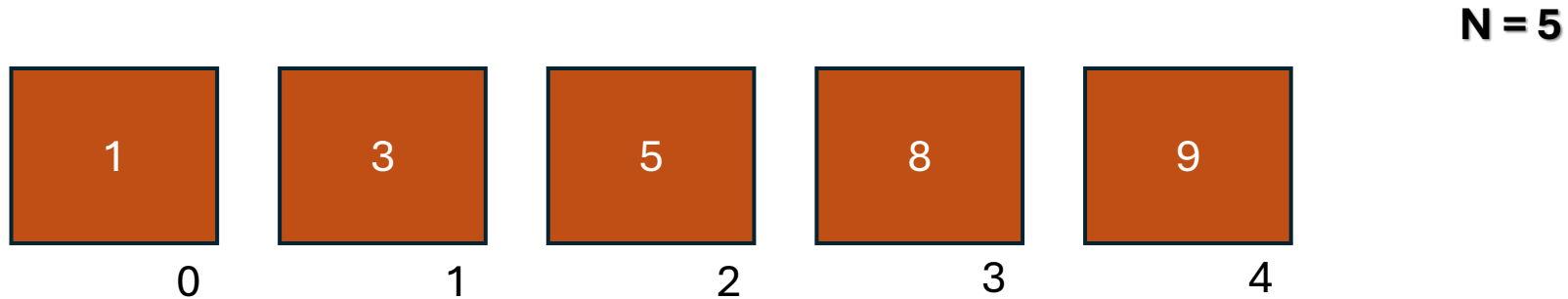
Comparación: 1

swaps: 1

Recorrido: 4

Ordenamiento por Intercambio

```
int vec[5] = { 3 , 9 , 1 , 8 , 5 };
```



Comparación: 4
swaps: 1
Recorrido: 1

Comparación: 3
swaps: 1
Recorrido: 2

Comparación: 2
swaps: 2
Recorrido: 3

Comparación: 1
swaps: 1
Recorrido: 4

En cada recorrido la cantidad de swaps es en el peor de los casos igual al número de comparaciones

Ordenamiento por Intercambio

```
1  /* Ordenamiento por intercambio */
2
3  #include <stdio.h>
4
5  int main()
6  {
7      int i, j ;
8      int aux ;
9      int vec[] = { 8 , 4 , 5 , 9 , 1 };
10     int length = sizeof (vec) / sizeof(vec[0]);
11
12     for ( i = 0 ; i < length - 1 ; i++ )
13     {
14         for (j=i+1 ; j < length ; j++)
15             if (vec[i]>vec[j])
16             {
17                 aux      = vec[i] ;
18                 vec[i]    = vec[j];
19                 vec[j]    = aux ;
20             }
21
22     for (i=0; i< length ; i++)
23         printf("%d\n", vec[i]);
24     return 0;
25 }
```

El primer bucle hace **n-1** comparaciones

El segundo bucle hace **n-i-1**

El número total de comparaciones forma **la sucesión matemática**: n-1, n-2, n-3, ... , 1
cuya suma es:

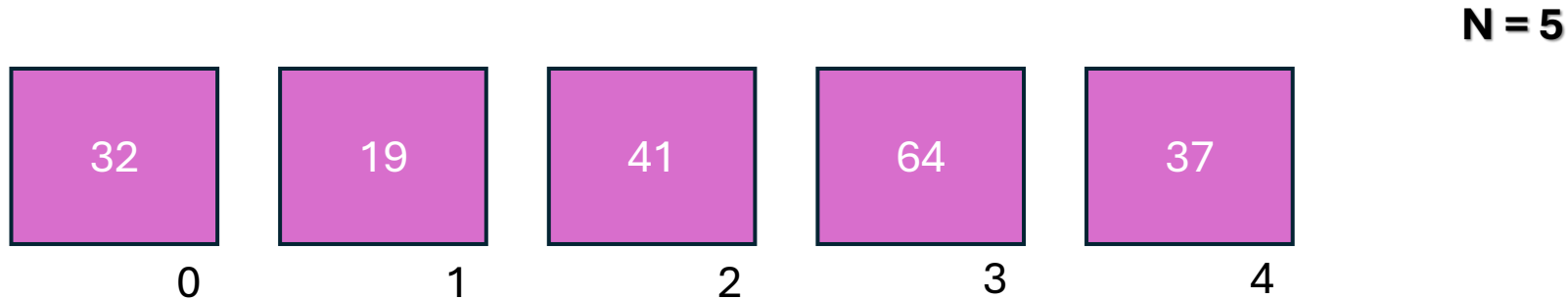
$$\frac{(n-1).n}{2}$$

El termino dominante es n^2 . Se dice que la Complejidad del método es cuadrática.

Poco eficiente cuando el conjunto a ordenar es grande

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



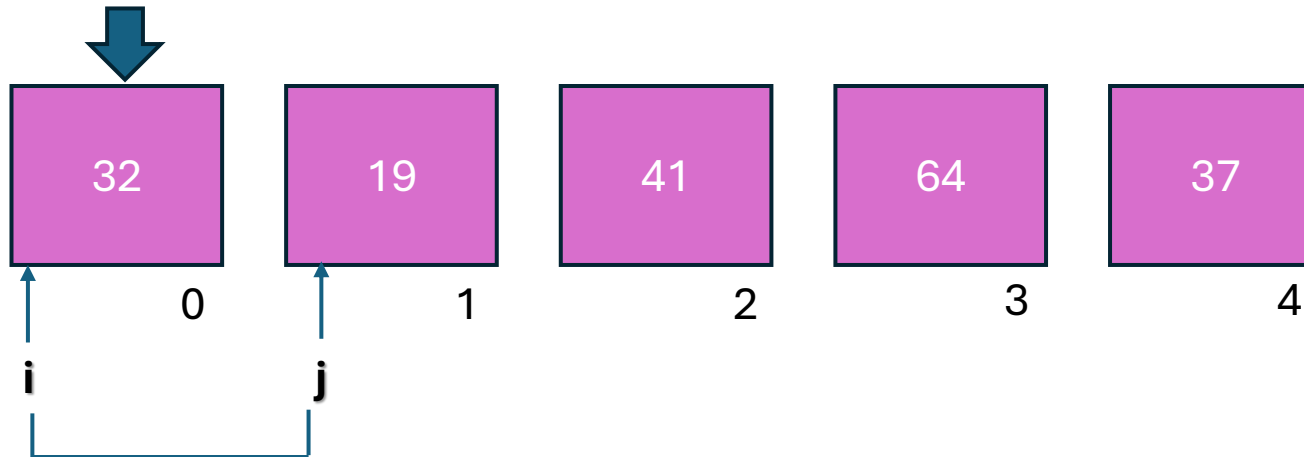
El método de selección ordena un vector encontrando en cada pasada el elemento más pequeño (o más grande, según el orden deseado) y colocándolo en su posición correcta.

En palabras simples, el algoritmo recorre el vector, busca el mínimo y lo intercambia con el elemento en la primera posición; luego busca el siguiente mínimo en el resto de la lista (posiciones 2 en adelante) y lo coloca en la segunda posición, y así sucesivamente.

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```

Candidato
a menor



N = 5

i comienza en cero

j comienza en i+1

- $19 < 32$
 - Candidato = 19

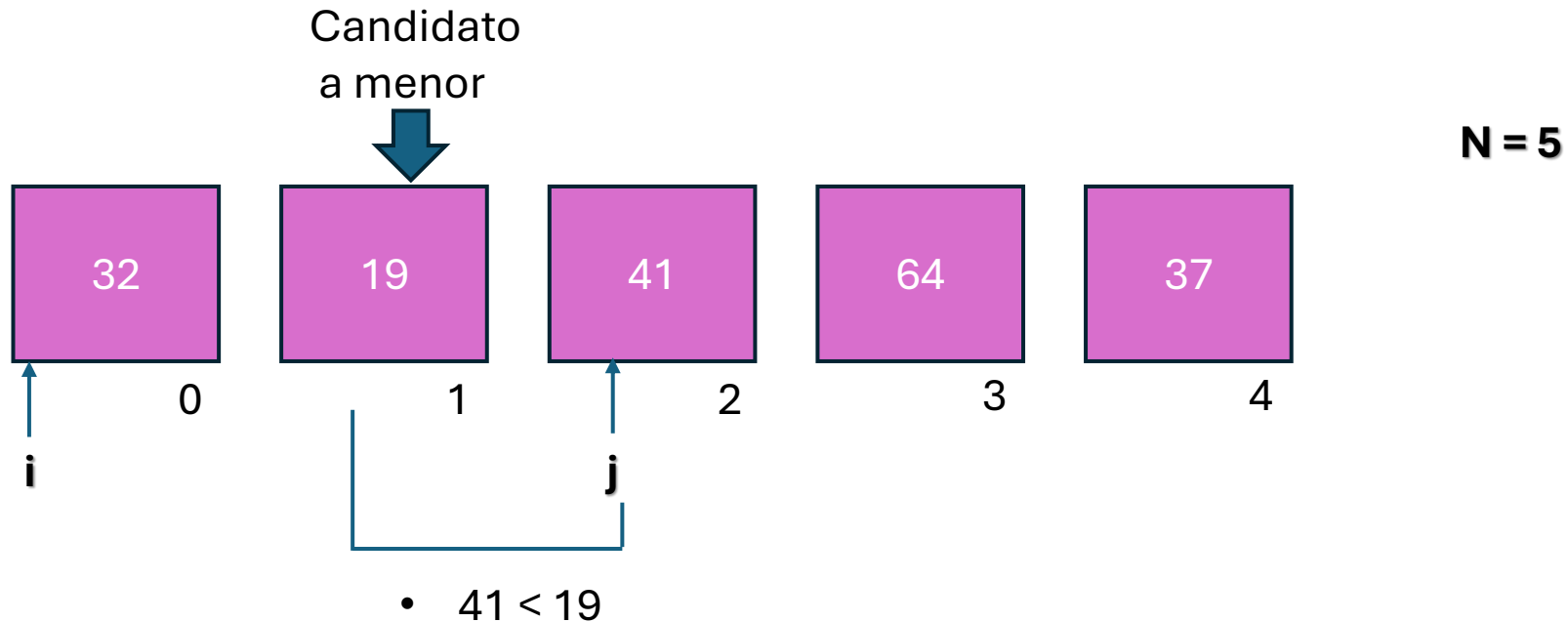
Comparación: 1

swaps: 0

Pasada: 1

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



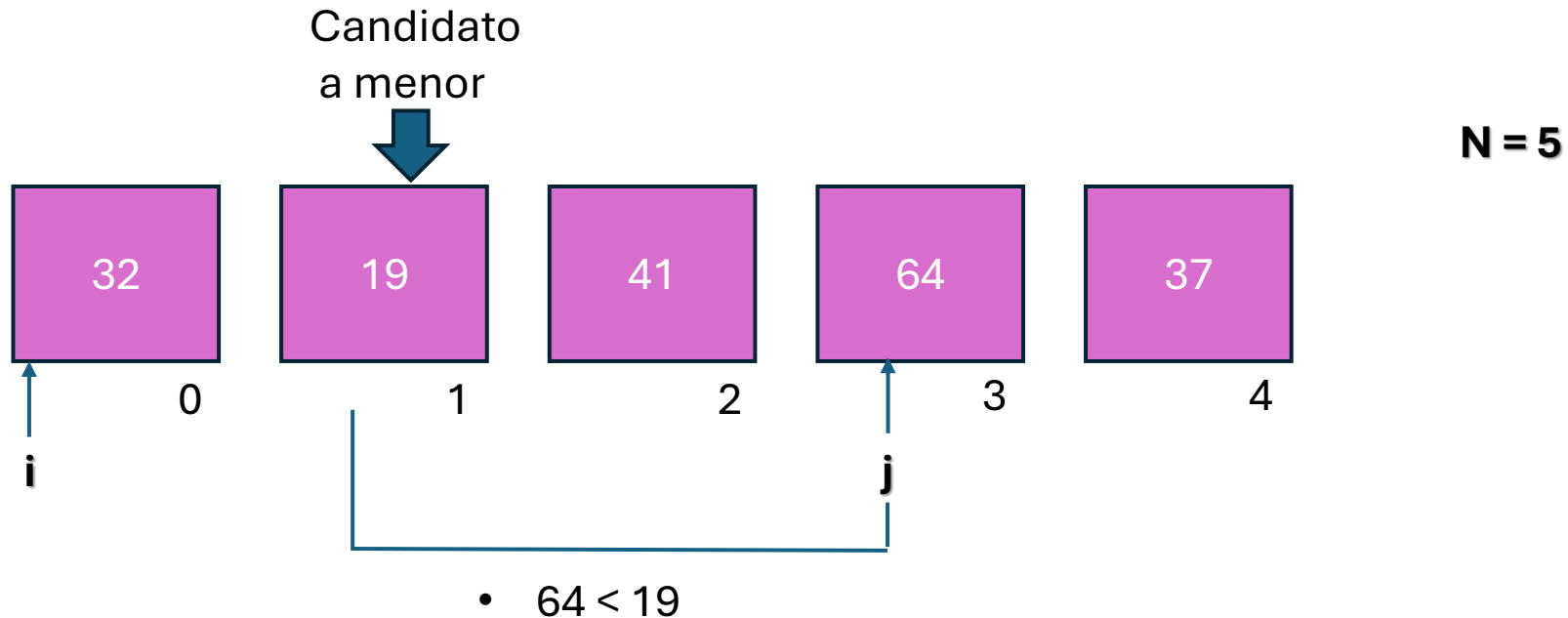
Comparación: 2

swaps: 0

Pasada: 1

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



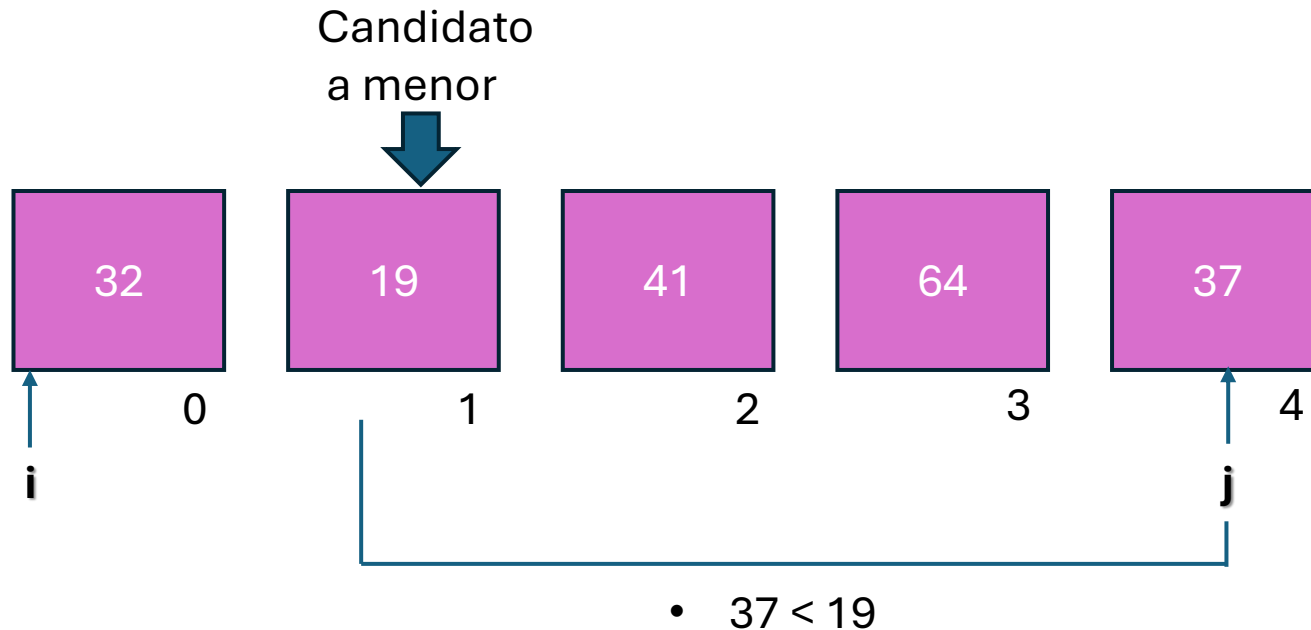
Comparación: 3

swaps: 0

Pasada: 1

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



$N = 5$

j recorre $N-1$ elementos

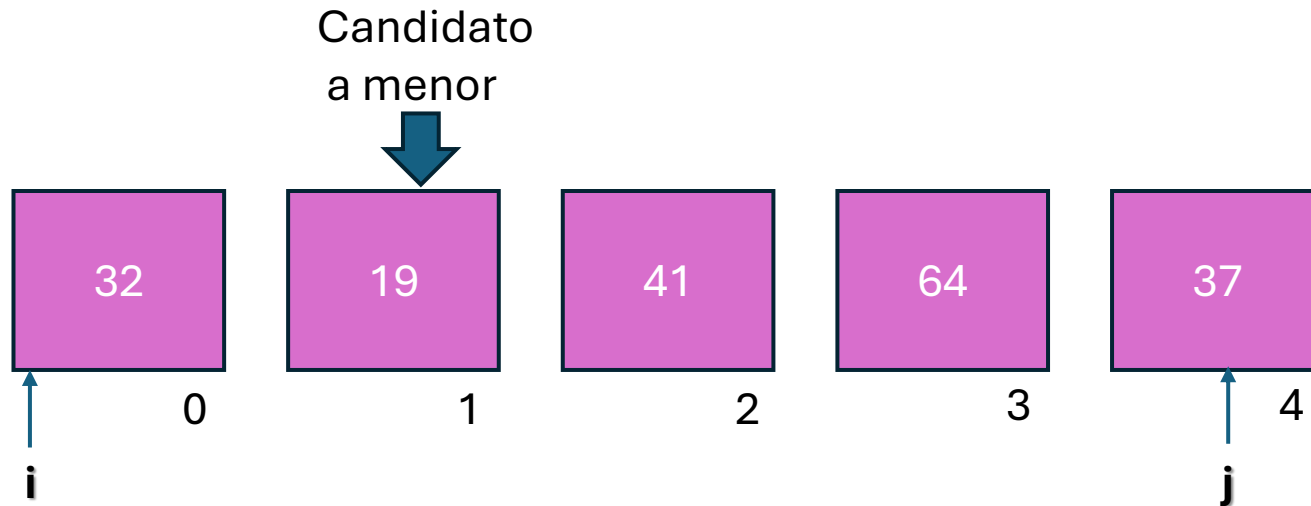
Comparación: 4

swaps: 0

Pasada: 1

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



N = 5

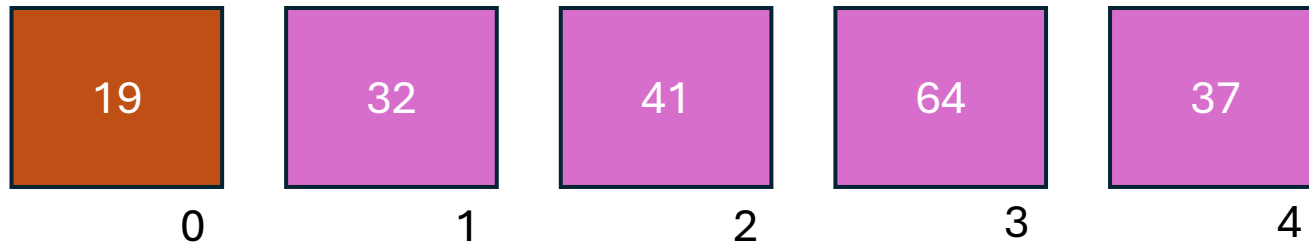
***j* recorre N elementos**

Comparación: 4
swaps: 0
Pasada: 1

Al finalizar el recorrido de *j*, se produce el UNICO SWAP de esta pasada.
Se lleva el candidato a la posición *i*.

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



N = 5

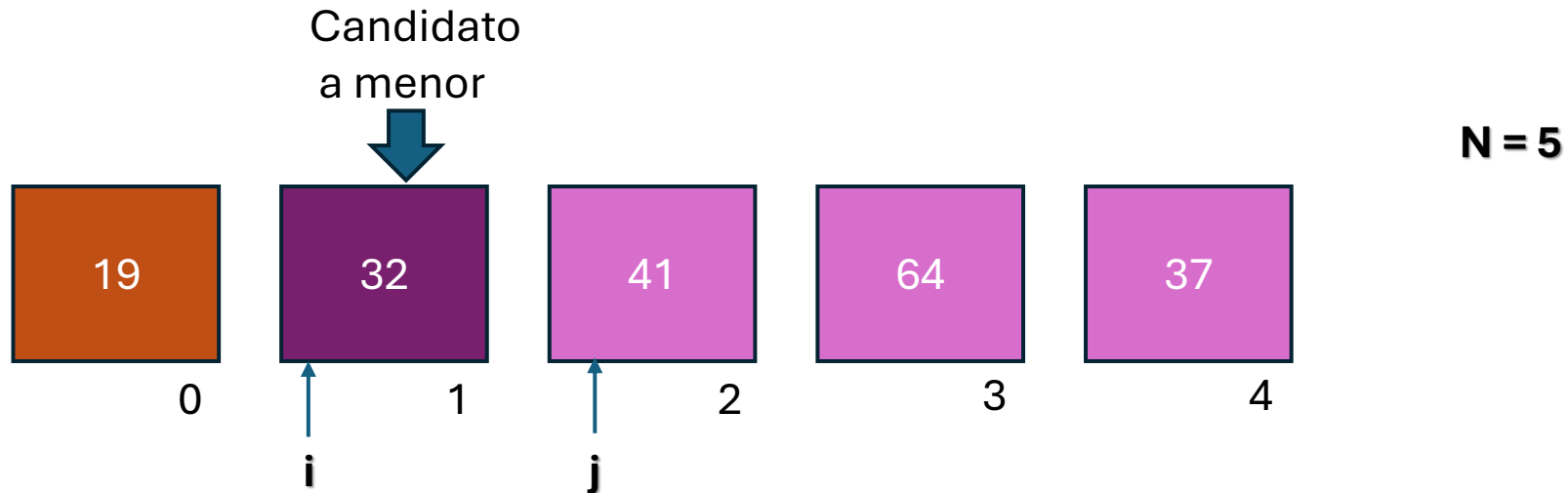
Comparación: 4

swaps: 1

Pasada: 1

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```

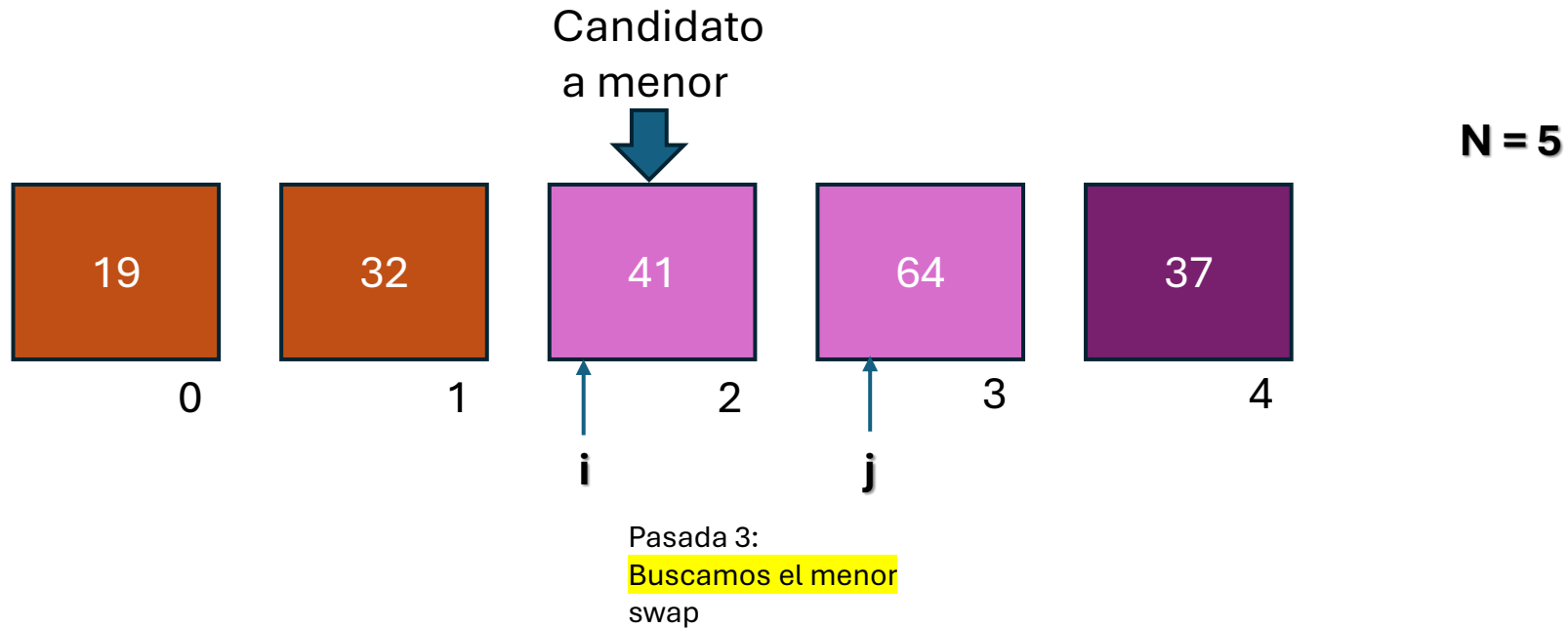


Pasada 2:
Buscamos el menor
swap

Comparación: 3
swaps: 0
Pasada: 2

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37};
```



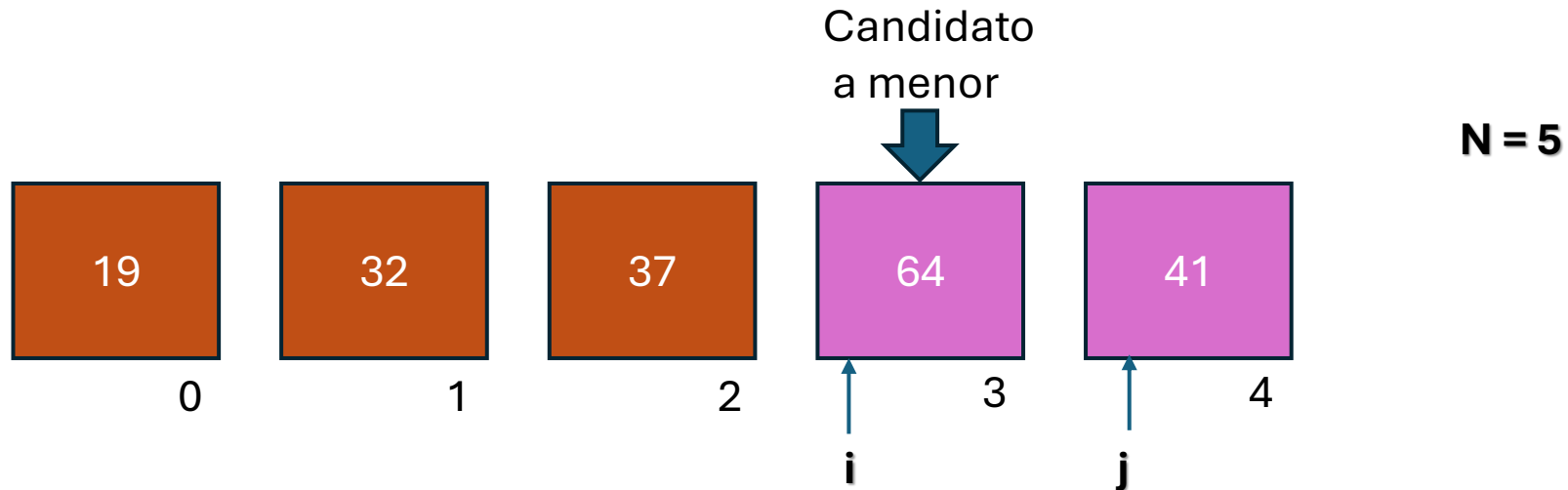
Comparación: 2

swaps: 0

Pasada: 3

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37};
```



Pasada 3:
Buscamos el menor
swap

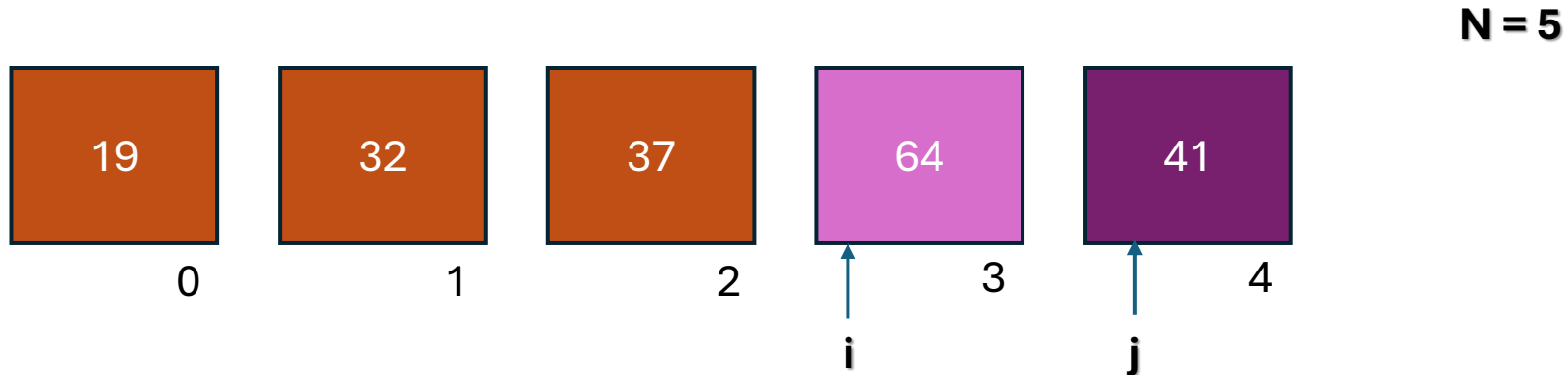
Comparación: 2

swaps: 1

Pasada: 3

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37};
```

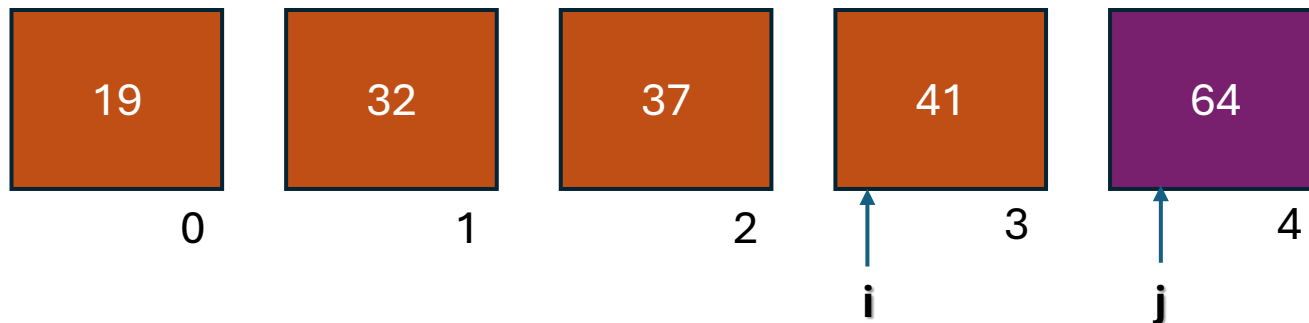


Pasada 4:
Buscamos el menor
swap

Comparación: 1
swaps: 0
Pasada: 4

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



$N = 5$

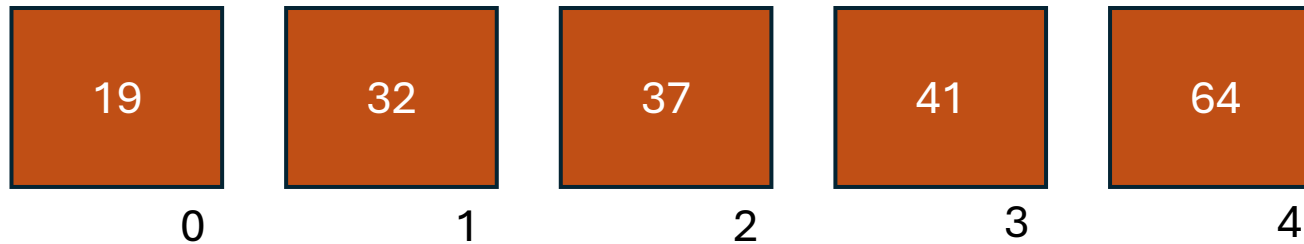
i recorre $N-1$ elementos
 j recorre $N-1-i$ elementos

Pasada 4:
Buscamos el menor
swap

Comparación: 1
swaps: 1
Pasada: 4

Ordenamiento por selección [Selection Sort]

```
int vec[5] = { 32 , 19 , 41 , 64 , 37 };
```



Comparación: 4
swaps: 1
Pasada: 1

Comparación: 3
swaps: 0
Pasada: 2

Comparación: 2
swaps: 1
Pasada: 3

Comparación: 1
swaps: 1
Pasada: 4

La máxima cantidad de swaps por cada pasada es 1

Ordenamiento por selección [Selection Sort]

```
1  /* Ordenamiento por selección */
2
3  #include <stdio.h>
4
5  int main()
6  {
7      int i, j, iMenor ;
8      int aux ;
9      int vec[] = { 8 , 4 , 5 , 9 , 1 };
10     int length = sizeof (vec) / sizeof(vec[0]);
11
12     for ( i = 0 ; i < length - 1 ; i++ )
13     {
14         iMenor = i ;
15         for (j=i+1 ; j < length ; j++)
16             if (vec[j] < vec[iMenor])
17                 iMenor = j ;
18
19         if (i != iMenor)
20         {
21             aux = vec[i] ;
22             vec[i] = vec[iMenor];
23             vec[iMenor] = aux ;
24         }
25     }
26
27     for (i=0; i< length ; i++)
28         printf("%d\n", vec[i]);
29     return 0;
30 }
```

Al igual que el método de intercambio, el primer bucle hace **$n-1$** comparaciones y el segundo **$n-i-1$** .

Por lo tanto al igual que antes, el número total de es:

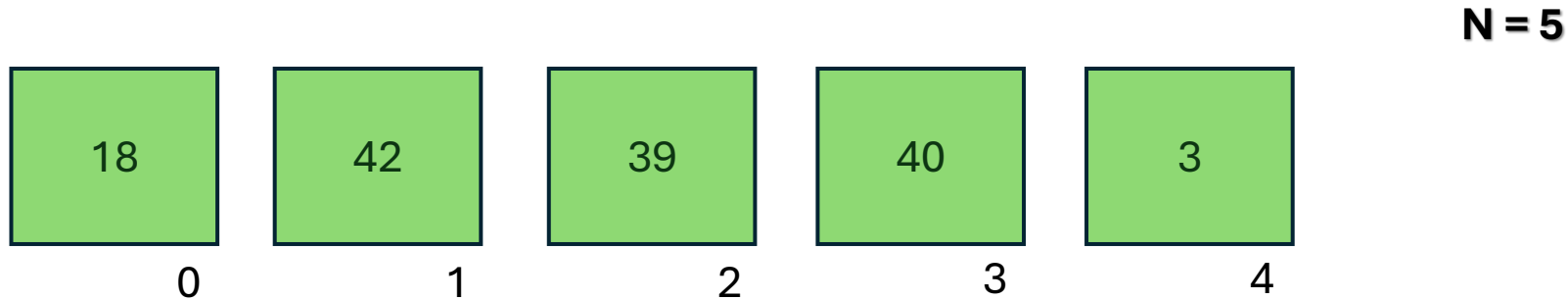
$$\frac{(n-1).n}{2}$$

El termino dominante es **n^2** . Por lo tanto la Complejidad del método sigue siendo **cuadrática**.

La única diferencia esta en el numero de swaps, en algunos casos este método podría ser mas eficiente que el de intercambio.

Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```

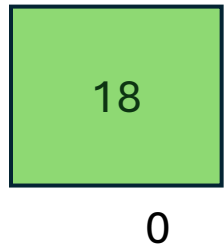
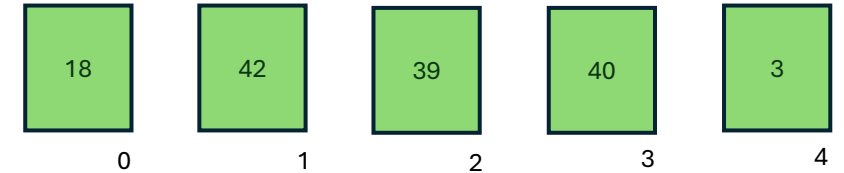


El **método de inserción** construye gradualmente el vector ordenado de la siguiente manera:

- Toma los elementos uno por uno y los va **insertando** en la posición adecuada dentro de la parte ya ordenada.
- Al inicio, se considera que el primer elemento ya está ordenado; luego se toma el segundo y se coloca en orden respecto al primero, luego el tercero respecto a los dos anteriores, y así sucesivamente.
- Este proceso es similar a como una persona ordenaría una mano de cartas: tomando cada carta nueva y ubicándola en el lugar correcto entre las que ya están ordenadas.

Ordenamiento por inserción [Insertion Sort]

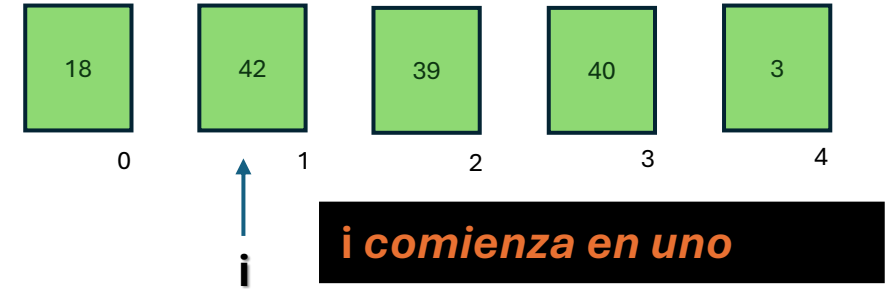
```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```



Comenzamos con el primer elemento. Lo consideramos ordenado.

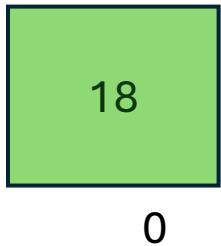
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```



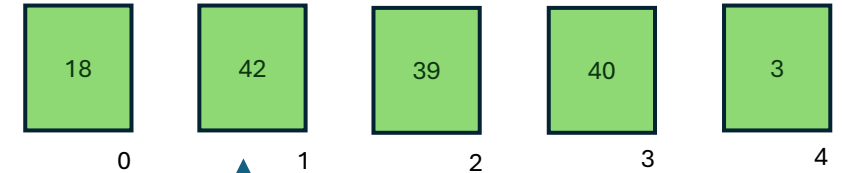
Procesamos 42:

Se inserta en la posición 1



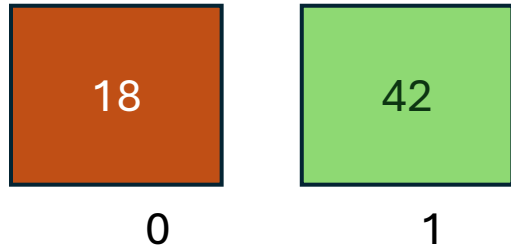
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```



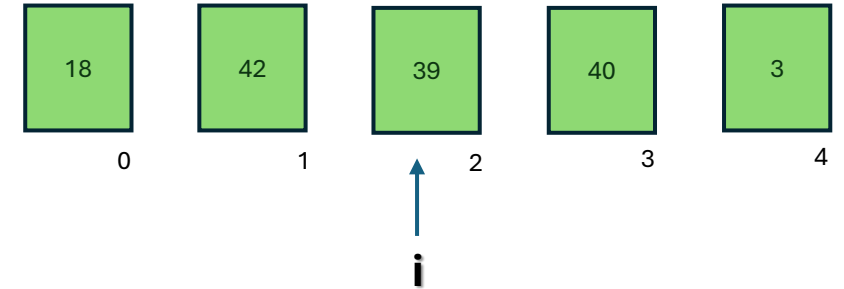
Procesamos 42:

Se inserta en la posición 1



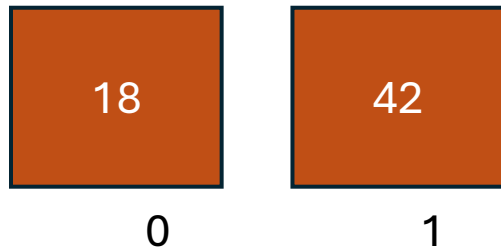
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```



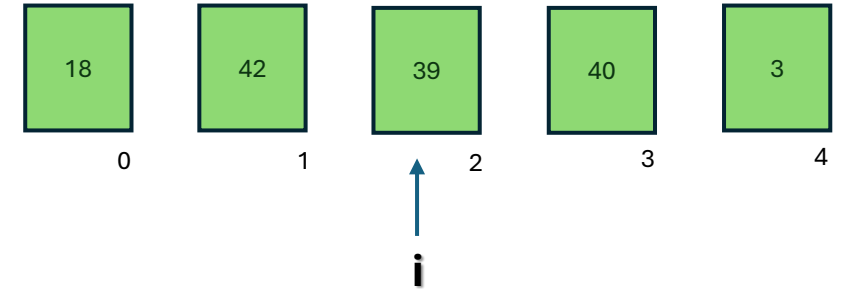
Procesamos 39:

Se inserta en la posición 1



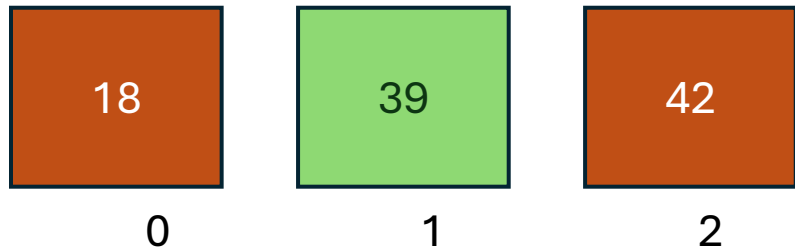
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3};
```



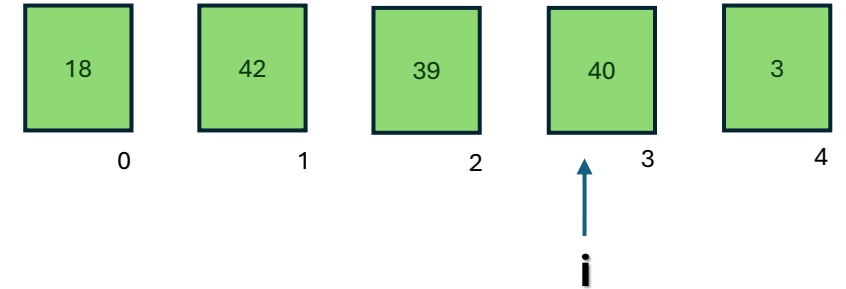
Procesamos 39:

Se inserta en la posición 1

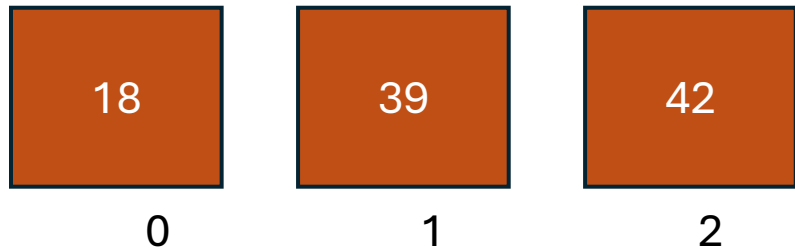


Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3};
```

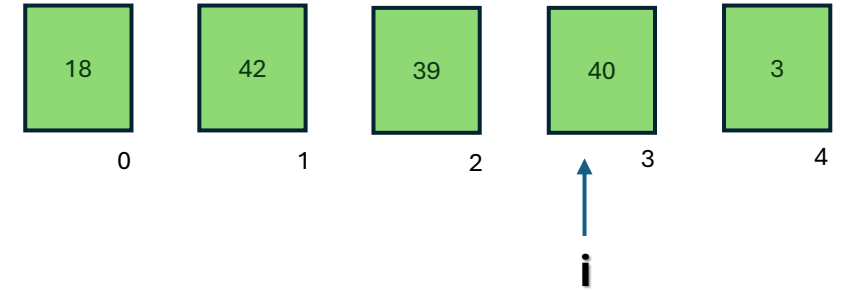


Procesamos 40:
Se inserta en la posición 2



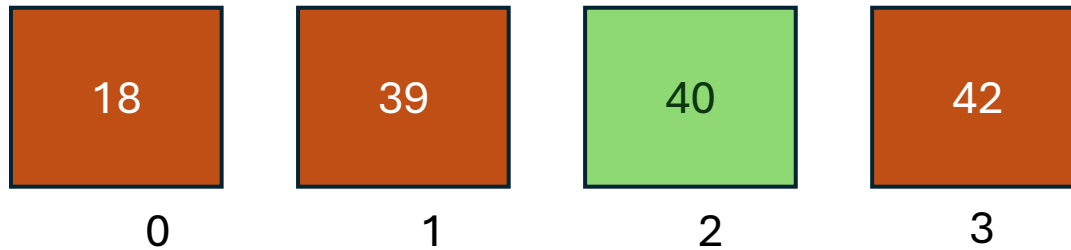
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```



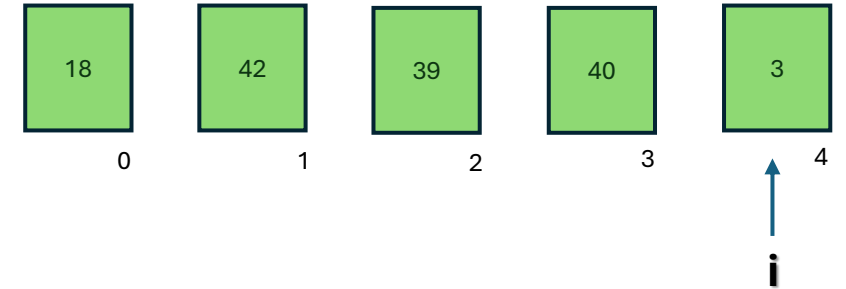
Procesamos 40:

Se inserta en la posición 2



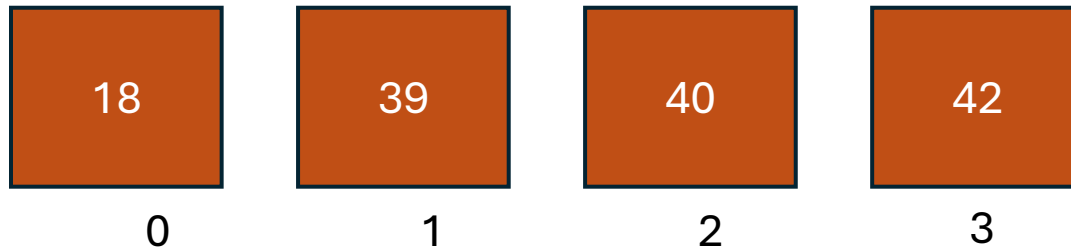
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3 } ;
```



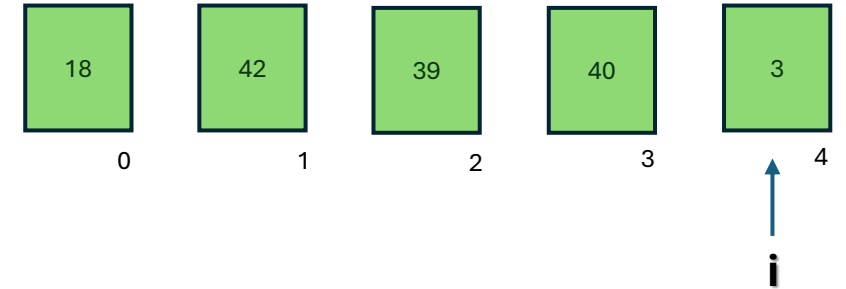
Procesamos 3:

Se inserta en la posición 0



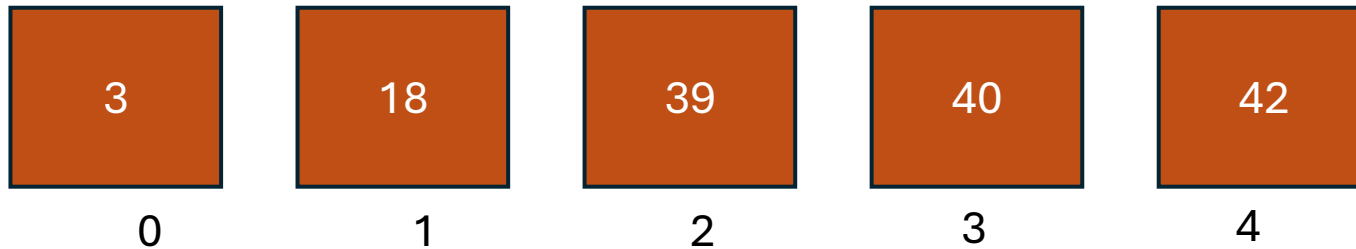
Ordenamiento por inserción [Insertion Sort]

```
int vec[5] = { 18 , 42 , 39 , 40 , 3};
```



Procesamos 3:

Se inserta en la posición 0



Ordenamiento por inserción [Insertion Sort]

```
1  /* Ordenamiento por insercion*/
2
3  #include <stdio.h>
4
5  int main()
6  {
7      int i, j ;
8      int aux ;
9      int vec[] = { 18 , 42 ,39 , 40 , 3 };
10     int length = sizeof (vec) / sizeof(vec[0]);
11
12     for ( i = 1 ; i < length ; i++ )
13     {
14         j = i ;
15         aux = vec[i];
16
17         while (j > 0 && aux < vec [j-1])
18         {
19             vec[j] = vec [j-1];
20             j--;
21         }
22         vec[j]=aux;
23     }
24
25     for (i=0; i< length ; i++)
26         printf("%d\n", vec[i]);
27     return 0;
28 }
29
```

El bucle principal corre **n-1** veces

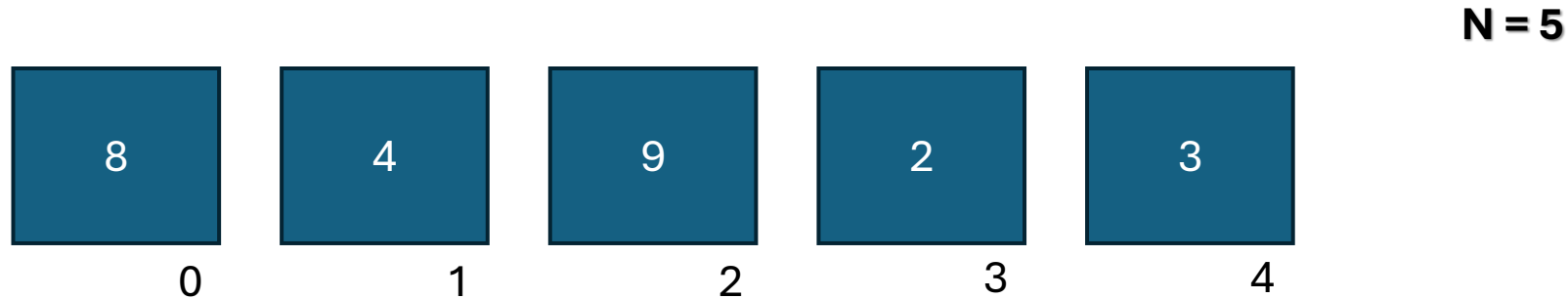
En el peor caso (vector invertido), por cada nuevo elemento el algoritmo recorre y desplaza muchos elementos del sub-vector ordenado, resultando en **$O(n^2)$ comparaciones y swaps.**

Quedándonos la misma complejidad que en los métodos anteriores: **complejidad cuadrática.** Pero por el numero de swaps hay que considerar:

- Método eficiente si el vector esta “casi ordenado”

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

```
int vec[5] = { 8 , 4 , 9 , 2 , 3 } ;
```

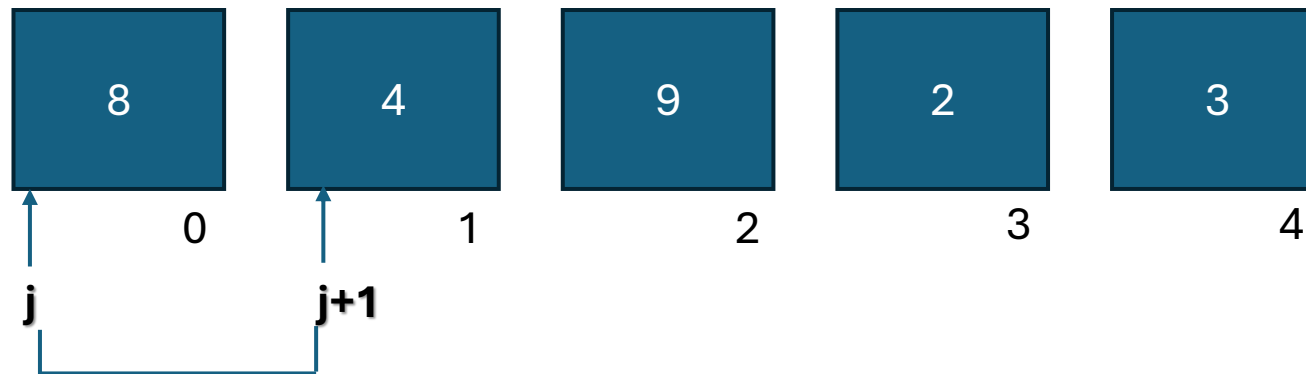


El **método de la burbuja** es quizás el más conocido de los algoritmos básicos. Funciona mediante **intercambios sucesivos de elementos adyacentes**: se comparan pares de elementos contiguos y se intercambian si están en orden equivocado. Con repetidas pasadas por el vector, los valores más grandes “flotan” hacia el final de la estructura, y los más pequeños se mueven hacia el inicio, de forma parecida a burbujas en un líquido.

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

La variable i no se usa para indexar a una posición del vector de forma directa. sino mas bien como un contador.

$i = 0$



$N = 5$

If (8 > 4)

intercambiar;

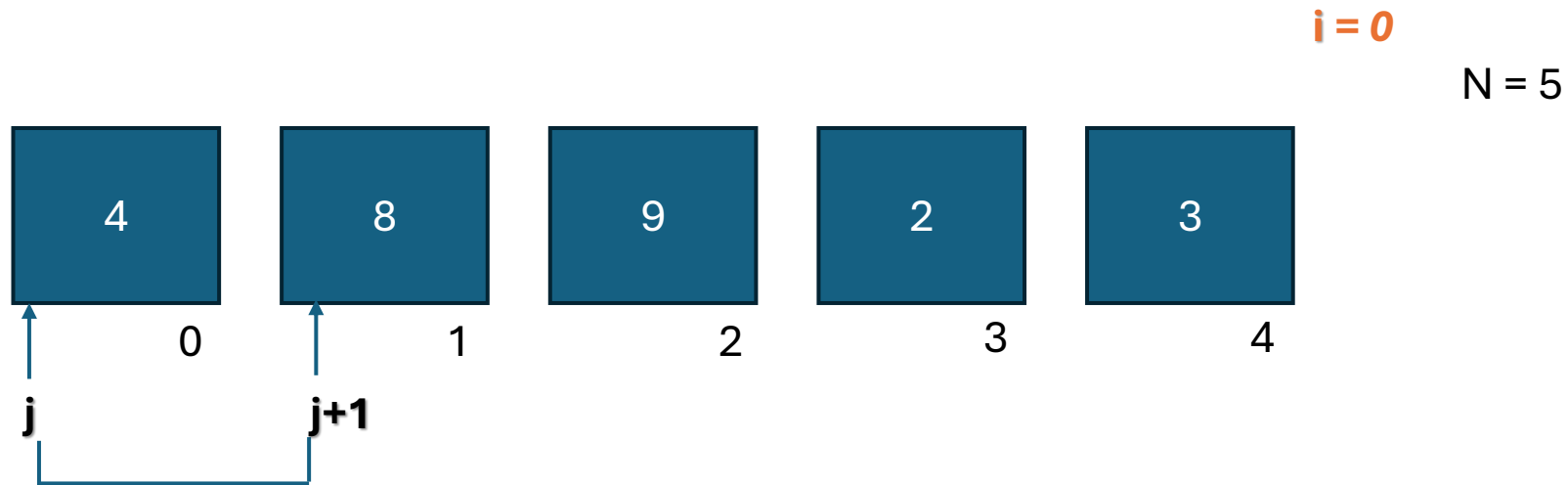
Comparación: 1

swaps: 0

Recorrido: 1

j comienza en 0

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



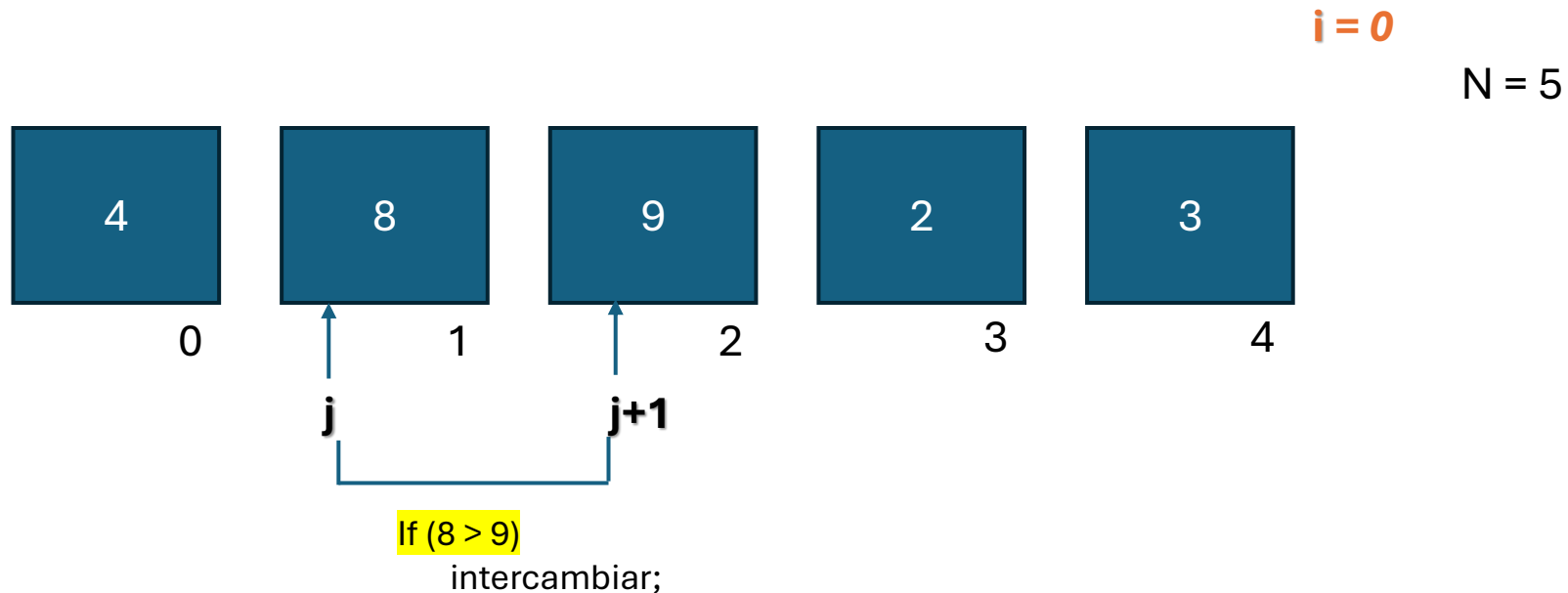
If ($8 > 4$)
intercambiar;

Comparación: 1

swaps: 1

Recorrido: 1

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

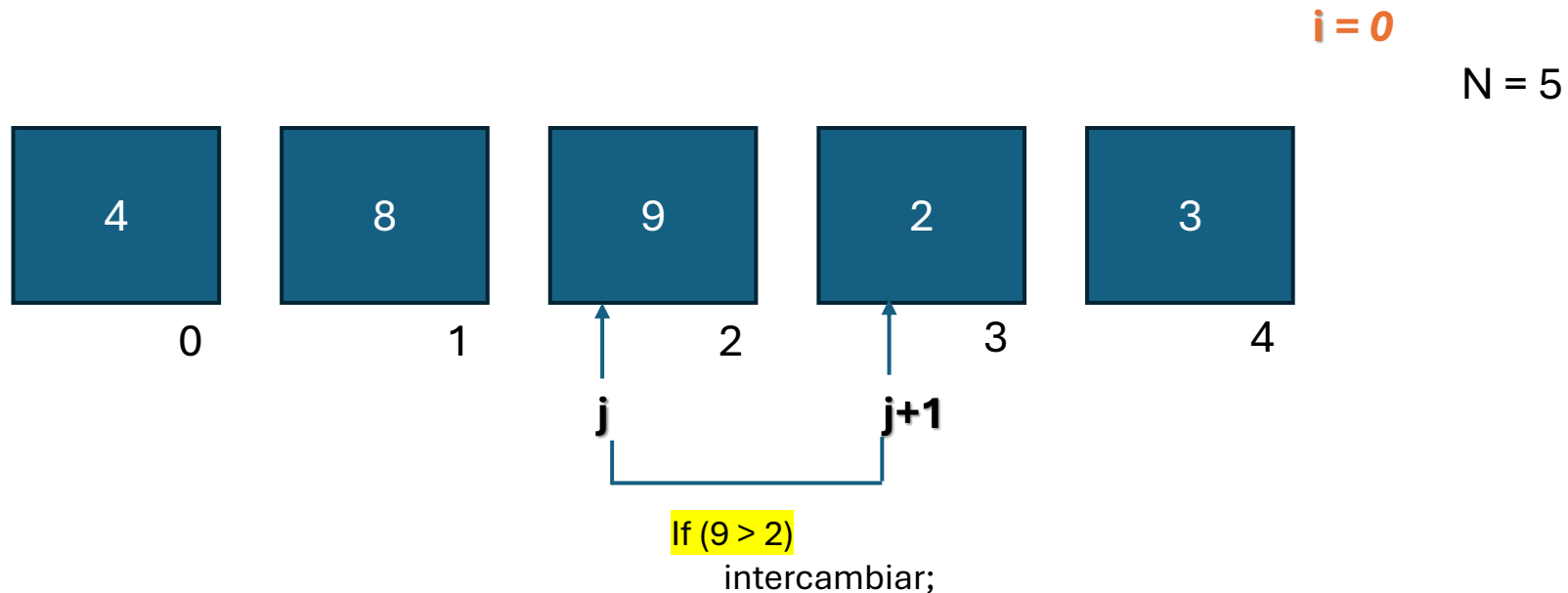


Comparación: 2

swaps: 1

Recorrido: 1

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

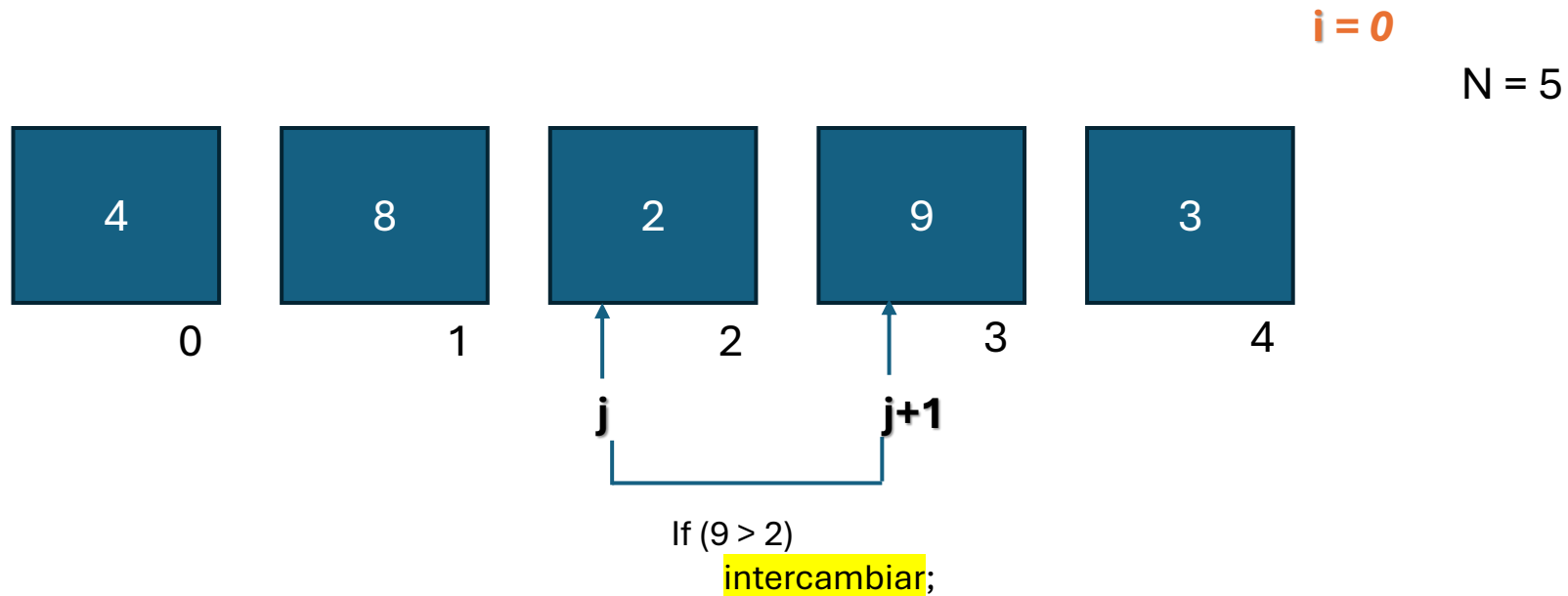


Comparación: 3

swaps: 1

Recorrido: 1

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

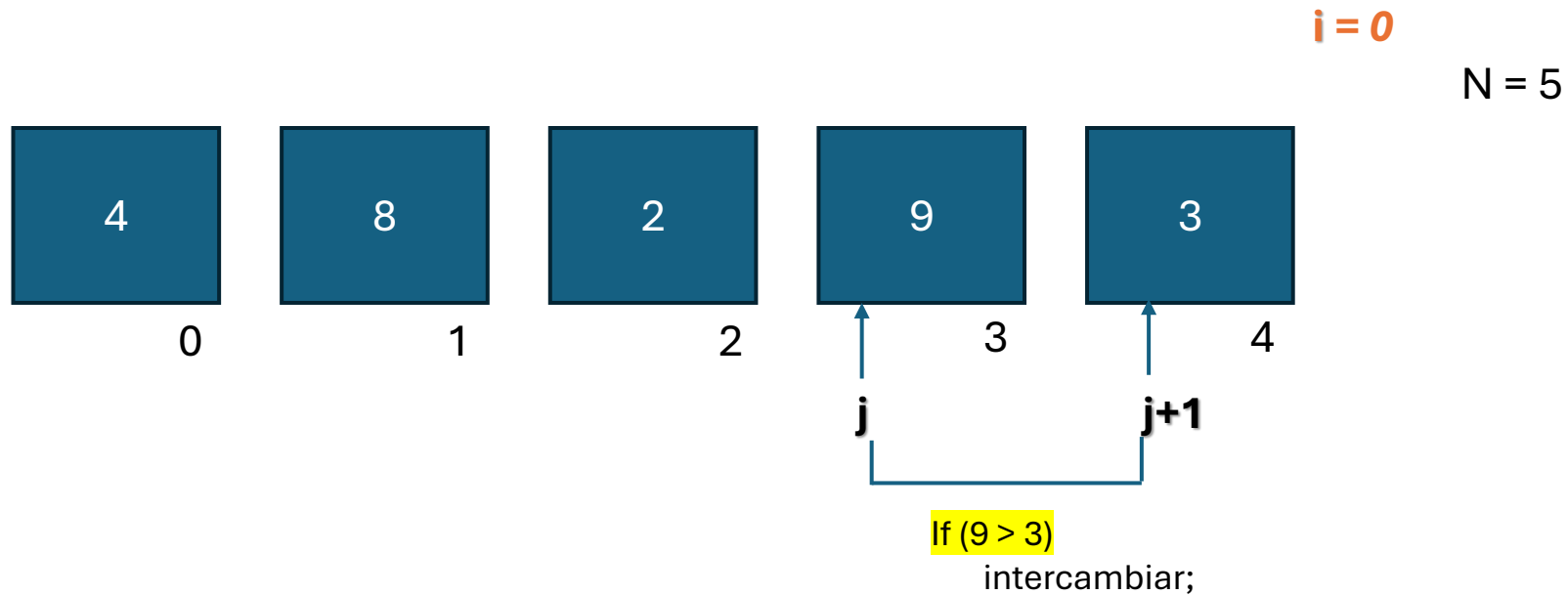


Comparación: 3

swaps: 2

Recorrido: 1

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



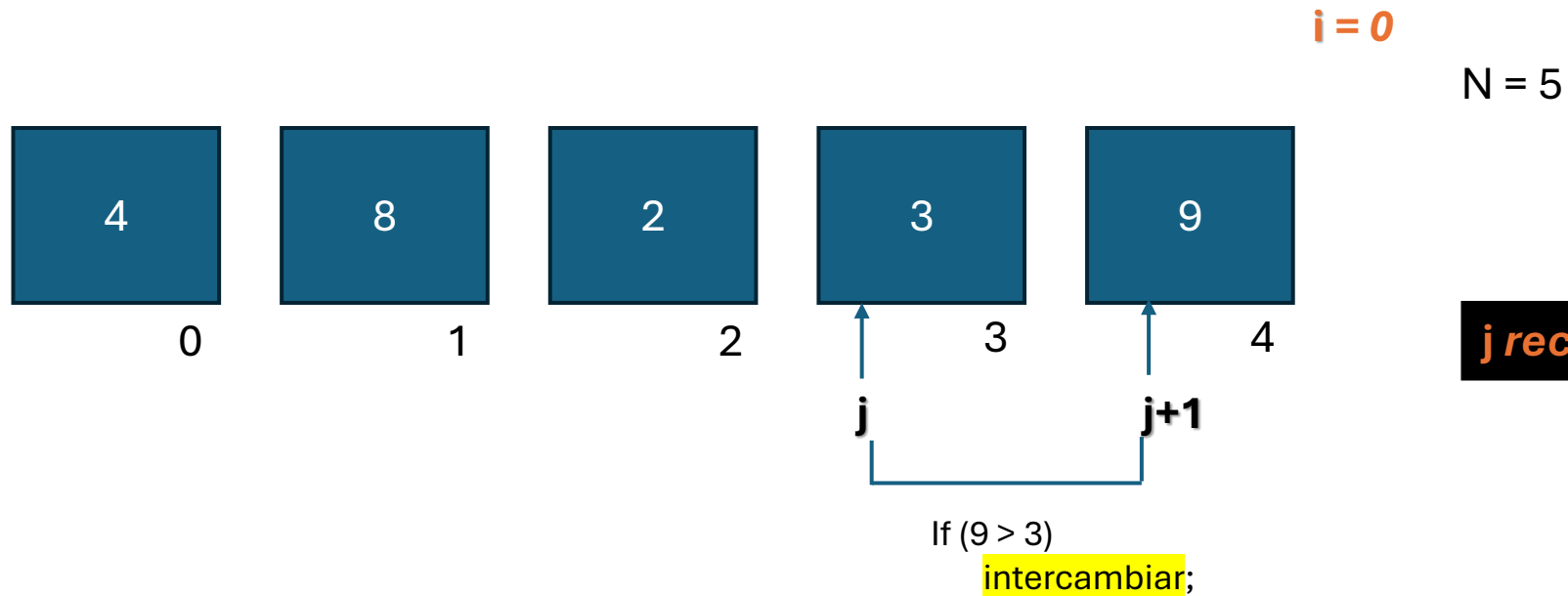
Comparación: 4

swaps: 2

Recorrido: 1

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

En cada pasada (i), los elementos más grandes van “burbujando” hacia el final del vector, y es j quien realiza las comparaciones e intercambios.



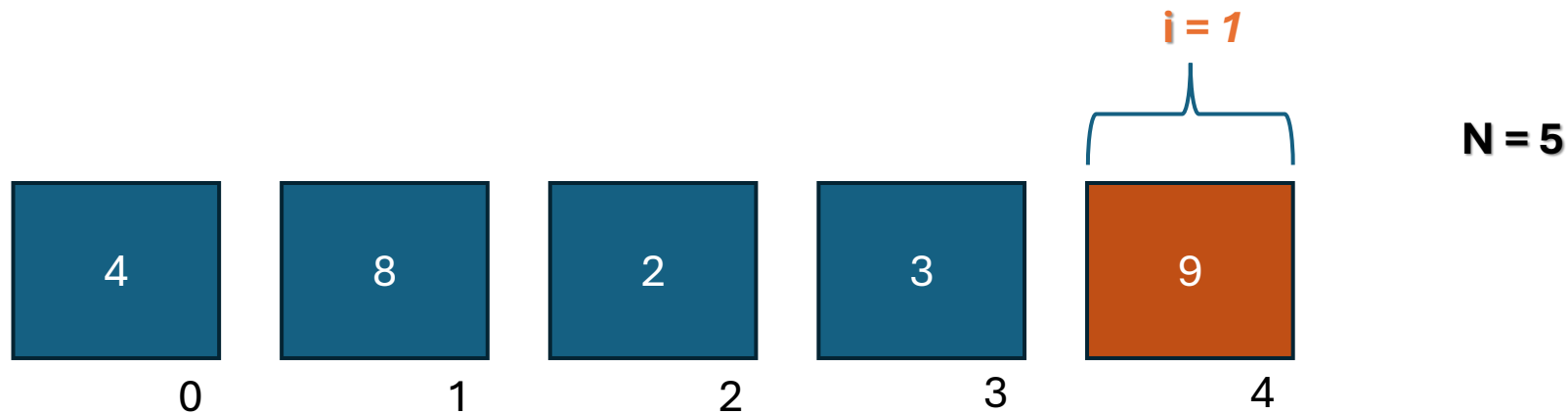
j recorre $N-1-i$ elementos

Comparación: 4

swaps: 3

Recorrido: 1

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

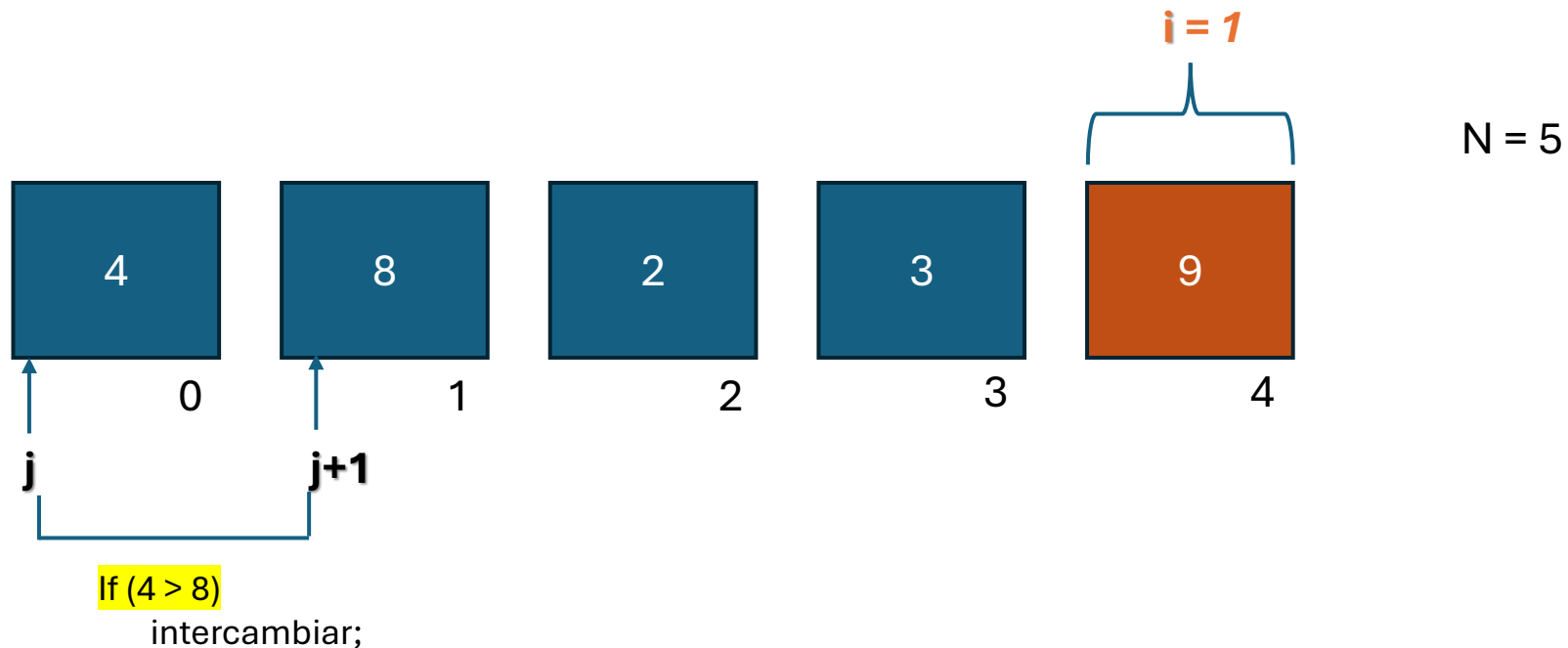


OBS: En caso de realizar todo el recorrido y no realizar ningún intercambio, quiere decir que la estructura ya estaba ordenada

Comparación: 4
swaps: 3
Recorrido: 1

En la primer vuelta se realizaron **(n-1)** comparaciones

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

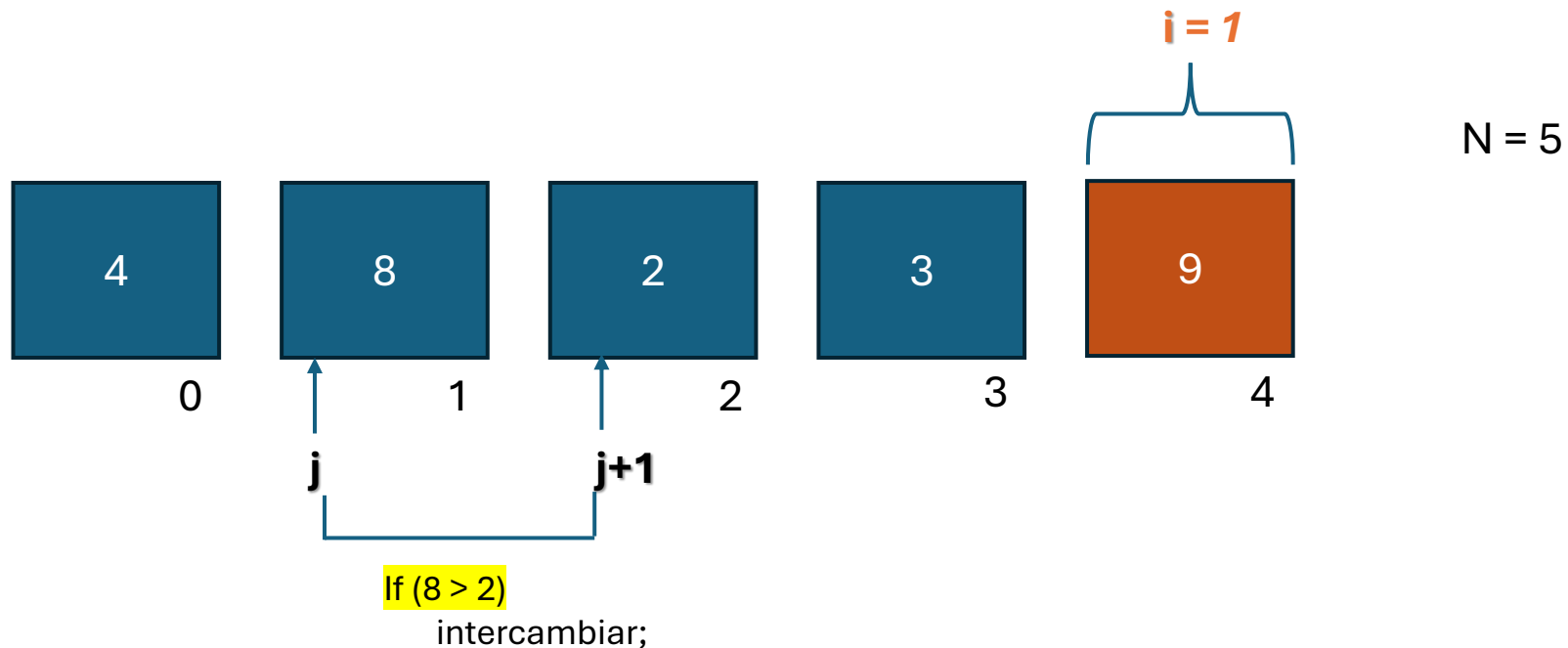


Comparación: 1

swaps: 0

Recorrido: 2

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

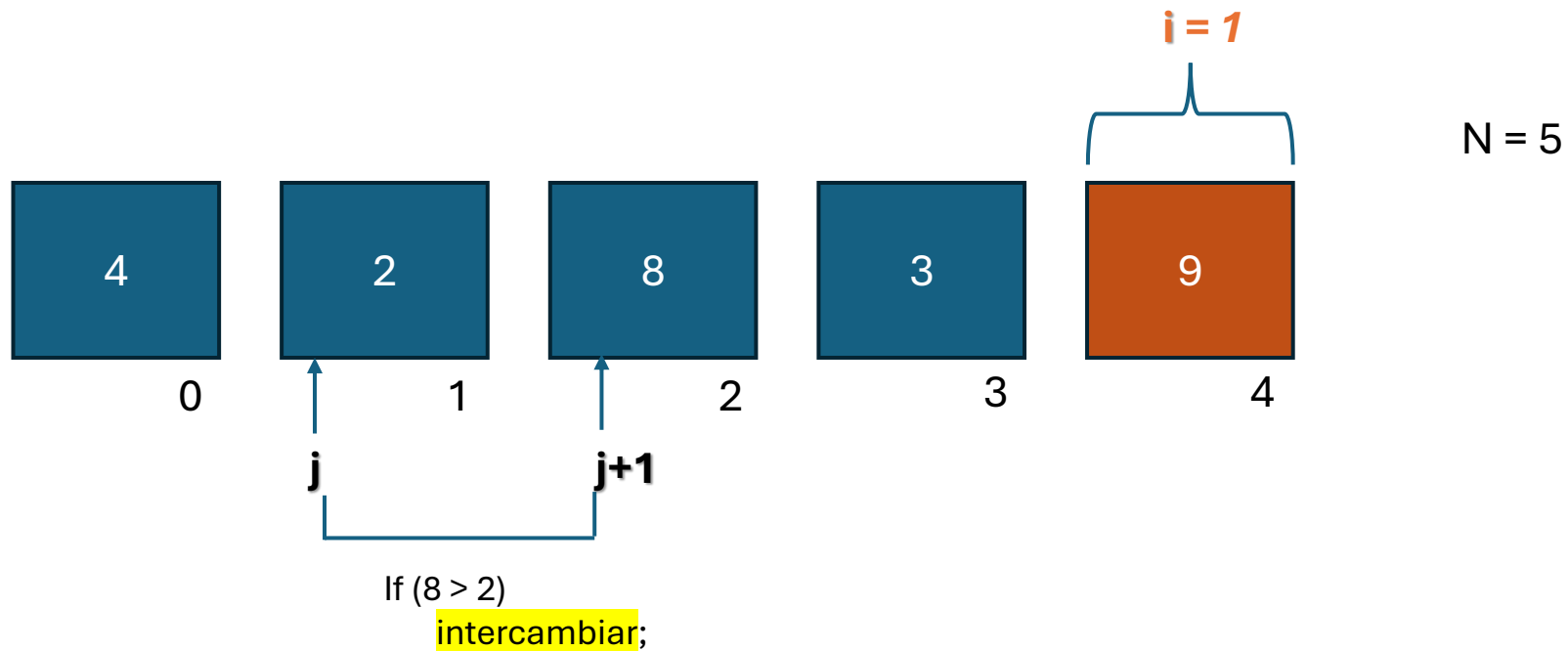


Comparación: 2

swaps: 0

Recorrido: 2

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

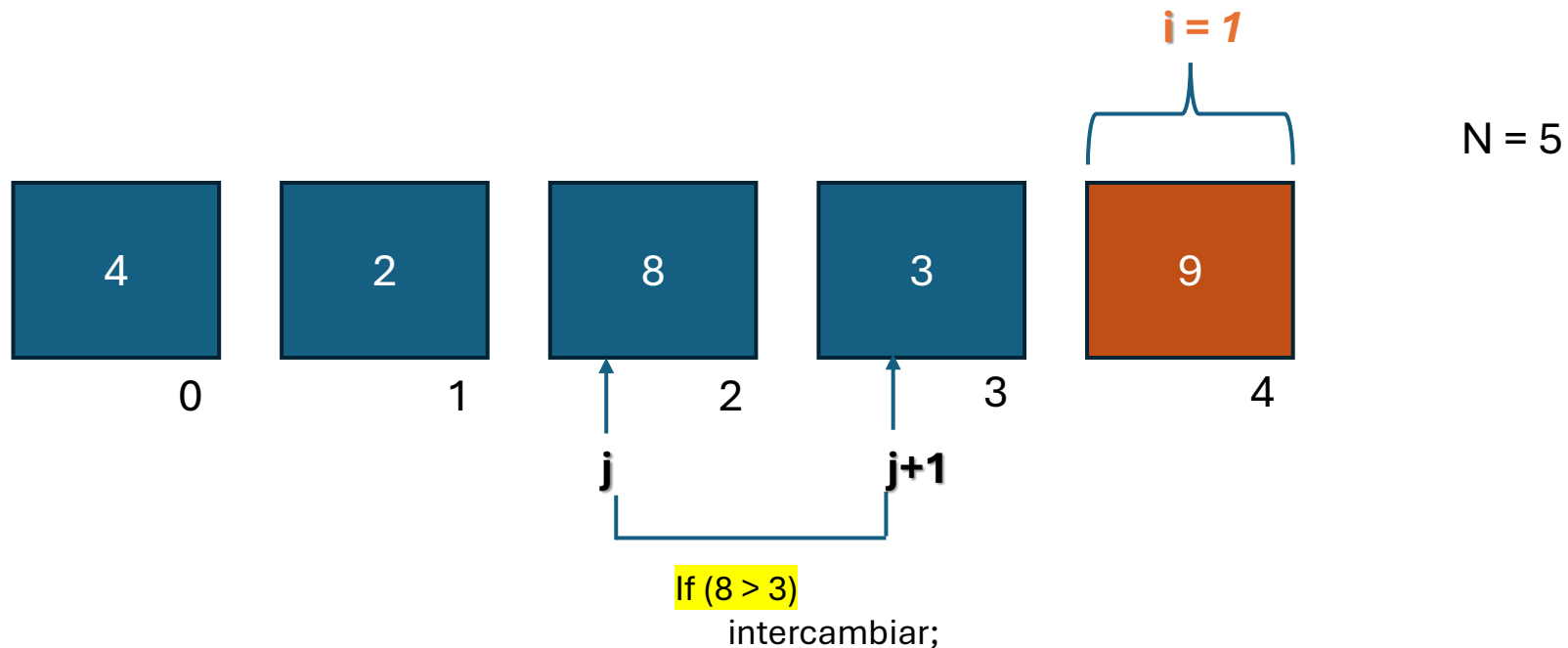


Comparación: 2

swaps: 1

Recorrido: 2

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

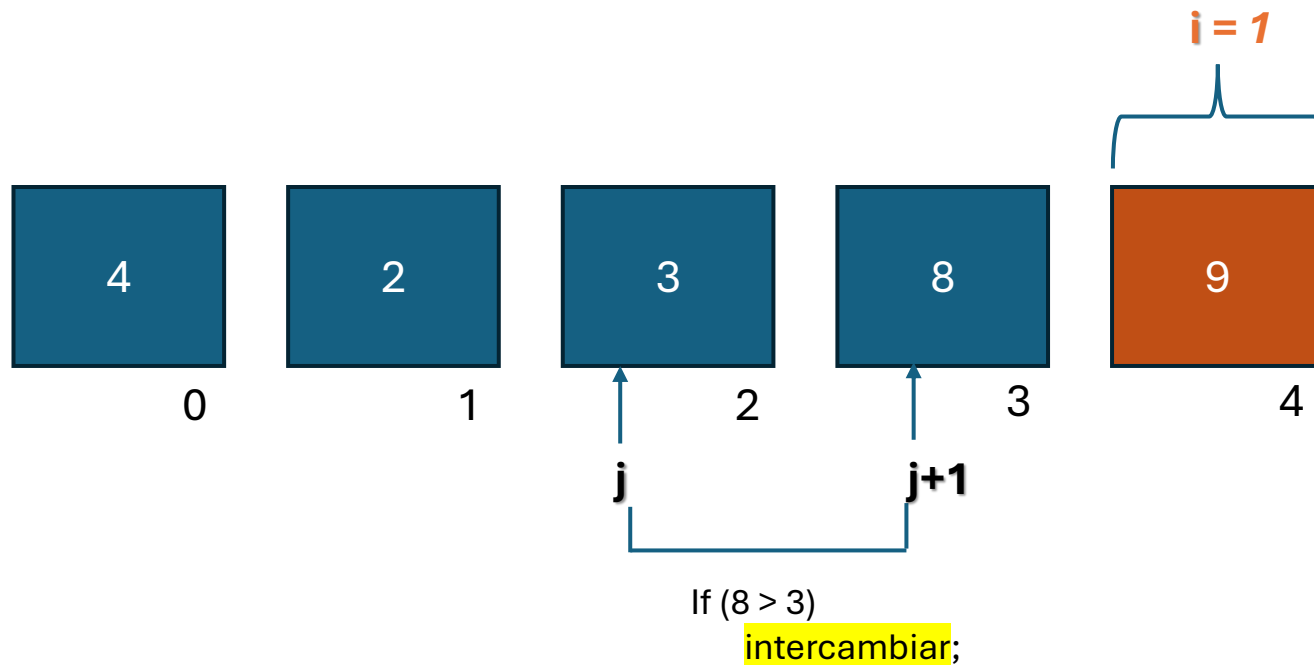


Comparación: 3

swaps: 1

Recorrido: 2

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



$N = 5$

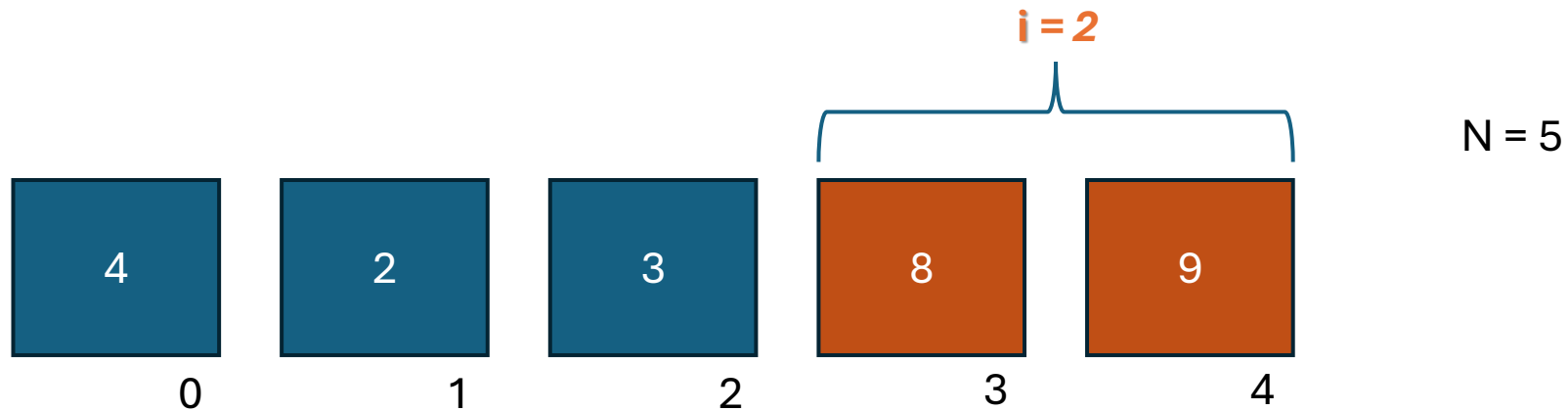
j recorre $N-1-i$ elementos

Comparación: 3

swaps: 2

Recorrido: 2

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



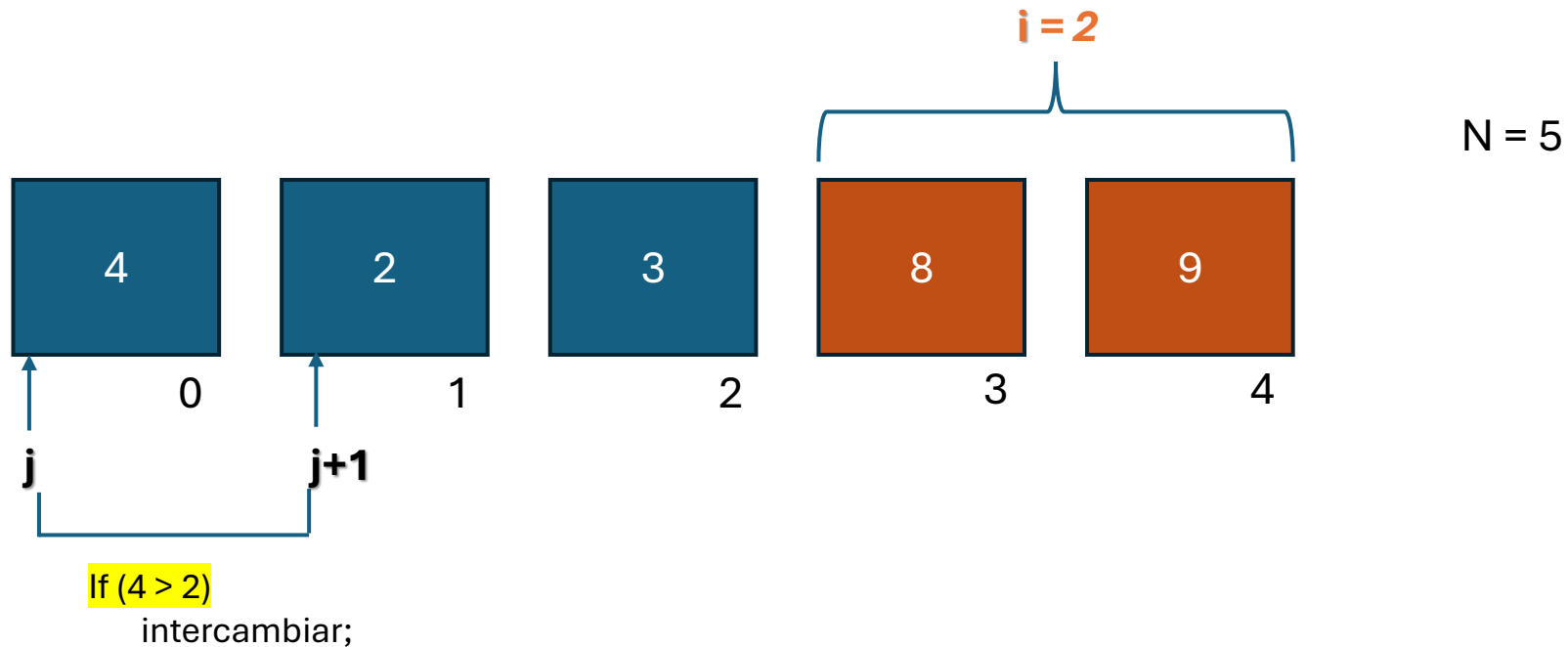
Comparación: 3

swaps: 2

Recorrido: 2

En la segunda vuelta se realizaron **(n-2)** comparaciones

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

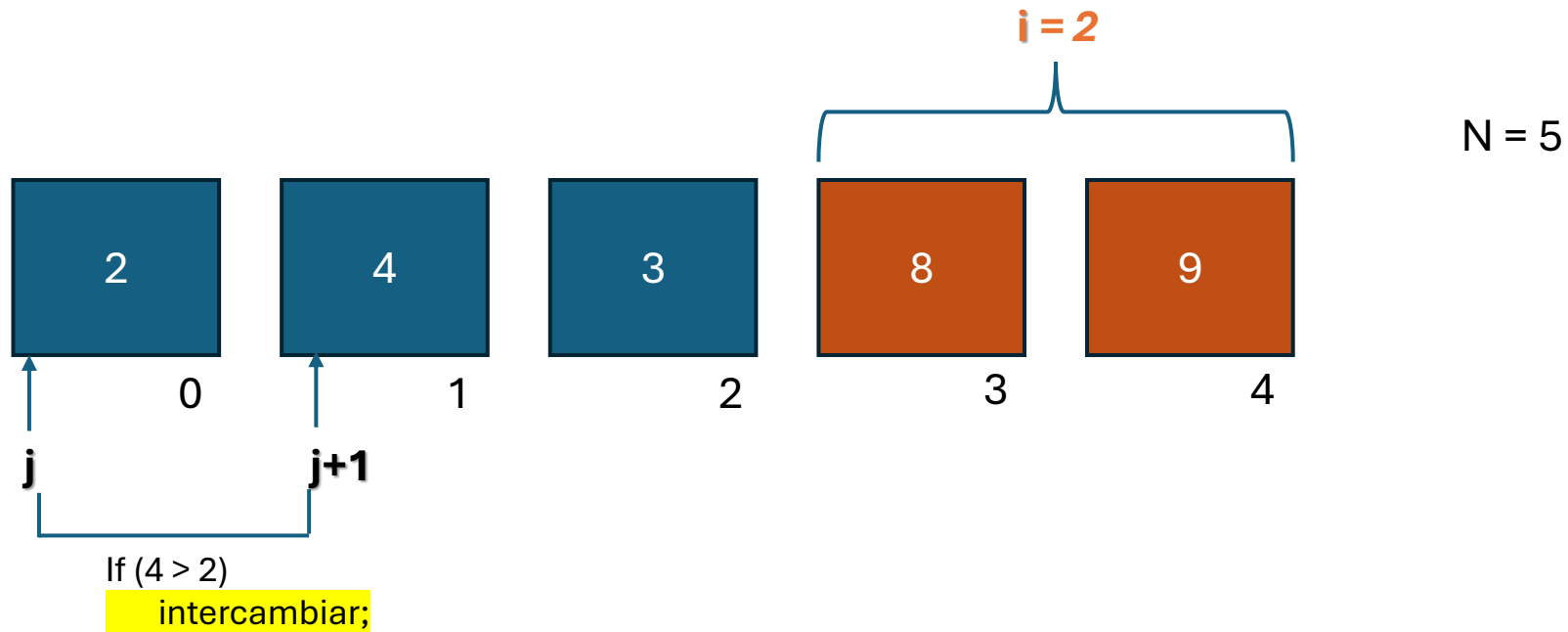


Comparación: 1

swaps: 0

Recorrido: 3

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

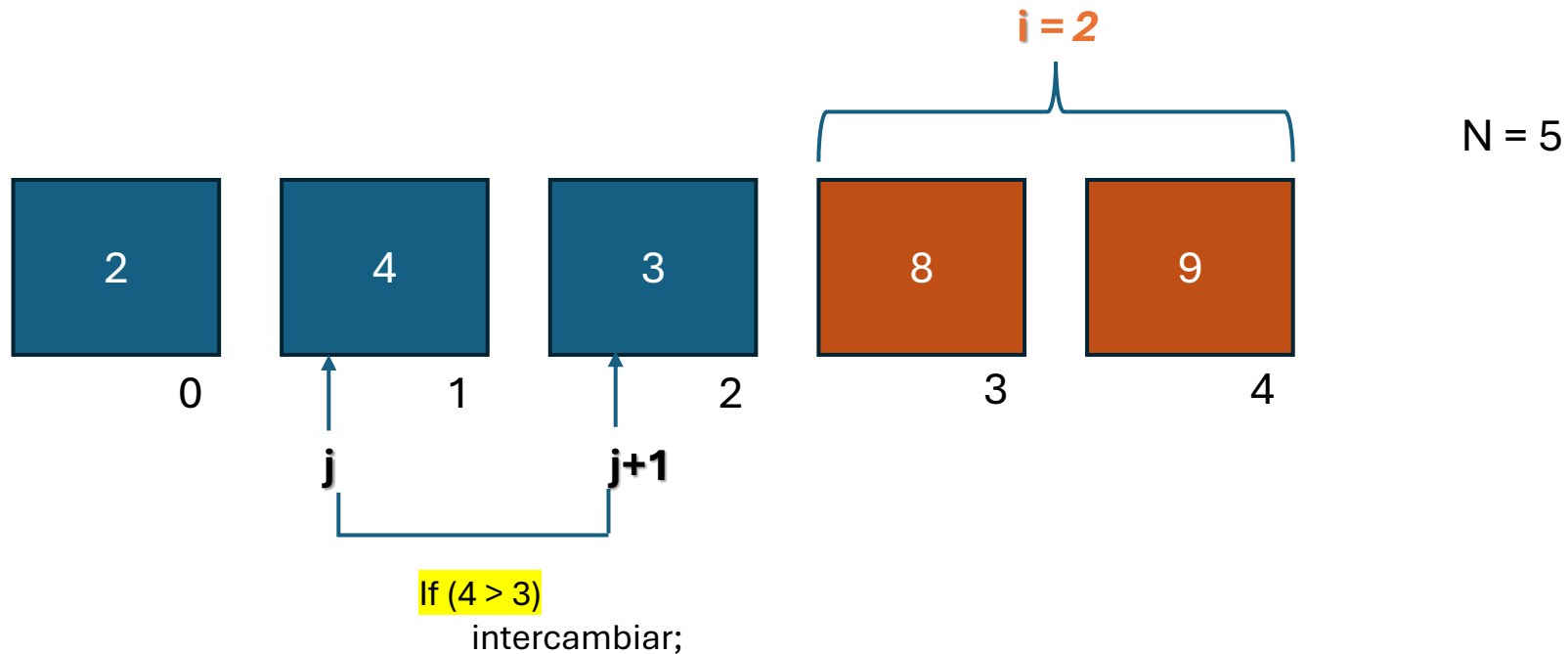


Comparación: 1

swaps: 1

Recorrido: 3

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

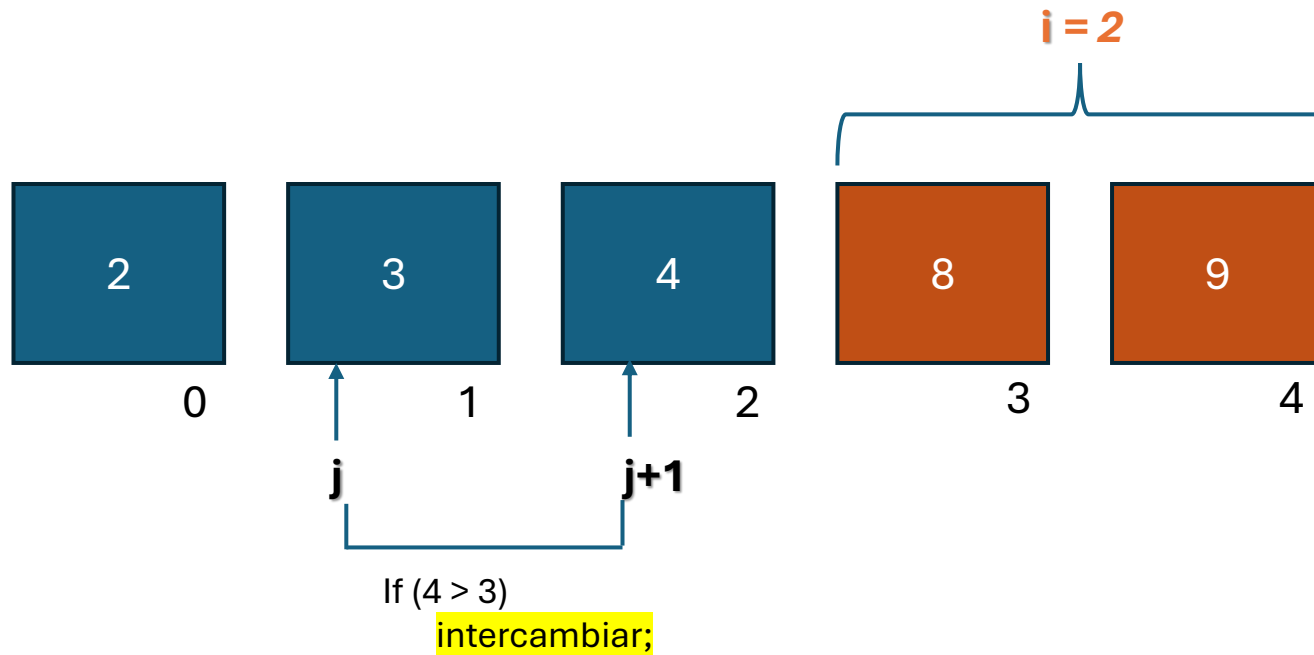


Comparación: 2

swaps: 1

Recorrido: 3

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



$N = 5$

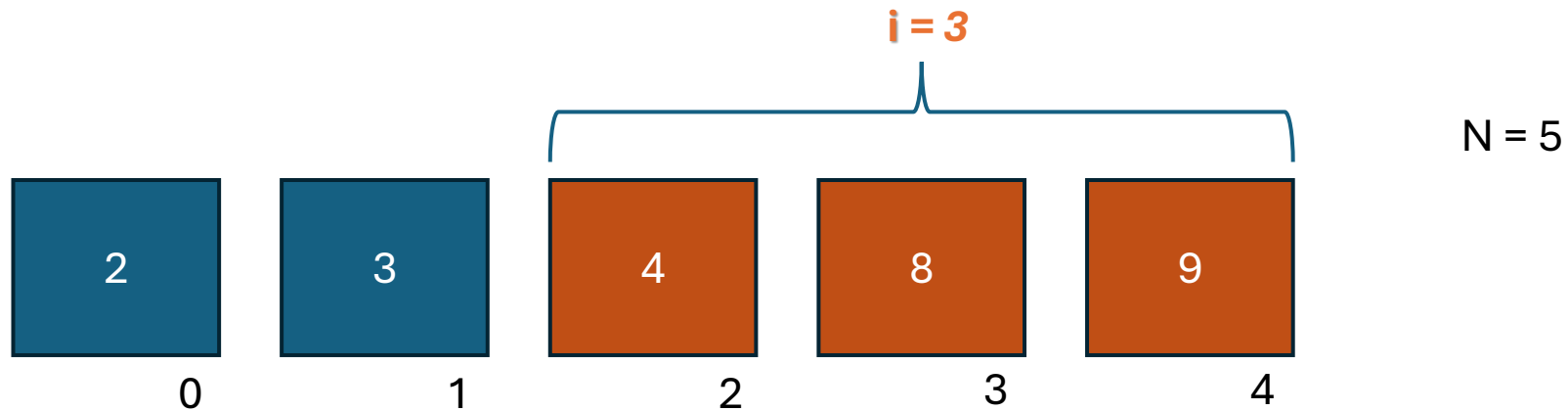
j recorre $N-1-i$ elementos

Comparación: 2

swaps: 2

Recorrido: 3

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



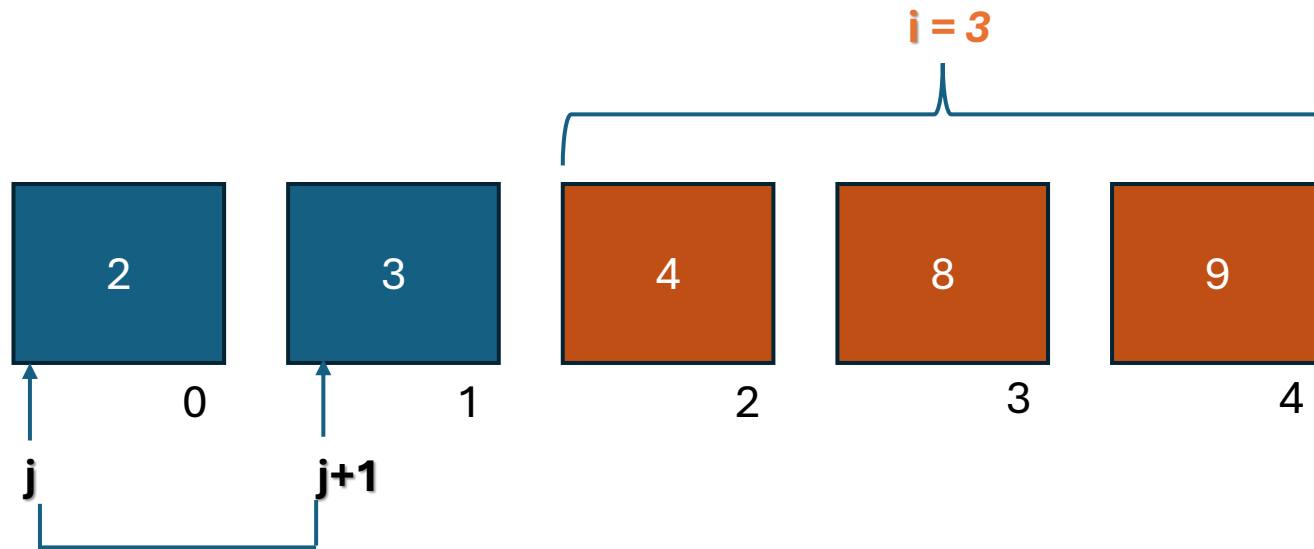
Comparación: 2

swaps: 2

Recorrido: 3

En la tercer vuelta se realizaron **$(n-3)$** comparaciones

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



If (2 > 3)
intercambiar;

Comparación: 1
swaps: 0
Recorrido: 4

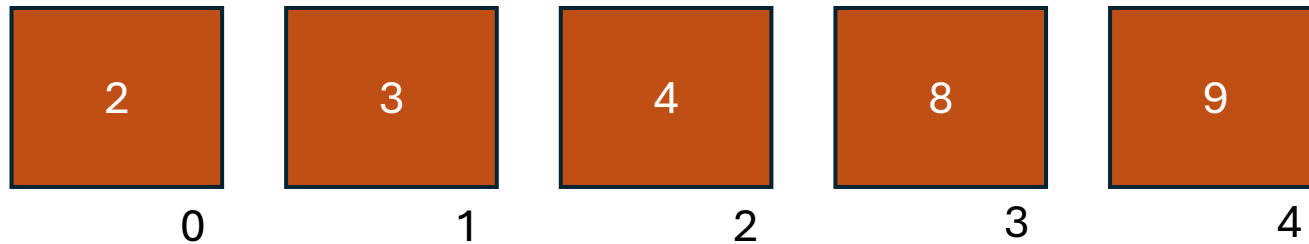
En la ultima vuelta se realizaron 1 comparación

$N = 5$

j recorre $N-1-i$ elementos

***i paso por 0,1,2,3
por lo tanto
i recorre $N-1$ elementos***

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]



Comparación: 4
swaps: 3
Recorrido: 1

Comparación: 3
swaps: 2
Recorrido: 2

Comparación: 2
swaps: 2
Recorrido: 3

Comparación: 1
swaps: 0
Recorrido: 4

$N = 5$

Comparaciones = 10

swaps = 7

Ordenamiento por burbujeo o intercambio directo [Bubble Sort]

```
1  /* Ordenamiento por burbujeo*/
2
3  #include <stdio.h>
4
5  int main()
6  {
7      int i, j ;
8      int aux ;
9      int vec[] = { 8 , 4 , 5 , 9 , 1 };
10     int length = sizeof (vec) / sizeof(vec[0]);
11
12     for ( i = 0 ; i < length - 1 ; i++ )
13         for (j=0 ; j < length - i - 1 ; j++)
14             if (vec[j]>vec[j+1])
15             {
16                 aux      = vec[j] ;
17                 vec[j]    = vec[j+1];
18                 vec[j+1] = aux ;
19             }
20
21     for (i=0; i< length ; i++)
22         printf("%d\n", vec[i]);
23     return 0;
24 }
25
```

Si la lista que ya está ordenada, el burbujeo simplemente pasa por ella una vez, así que su complejidad es n (lineal). Pero si estamos hablando del peor caso, necesitamos hacer $(n - i - 1)$ comparaciones e intercambios en cada iteración, y para ordenar toda la lista, eso suma un total de:

$$\frac{(n - 1).n}{2}$$

comparaciones y una cantidad similar de intercambios. Así que, en el peor caso, la complejidad del burbujeo también es cuadrática.

Si bien pueden ver casos optimistas, el método de burbujeo suele requerir más swaps que el método de selección. En general, el burbujeo tiende a ser más lento, especialmente con listas grandes.



TERMINAMOS CON
LOS METODOS
BASICOS